amc_dsl_text - Payoff Language Reference

Operators, keywords, and the complete built-in function vocabulary for writing `.amc` payoffs

amc_engine / amc_dsl_text

June 2026

amc_dsl_text - Payoff Language Reference

This document is an exhaustive reference for **everything you can type inside a `.amc` payoff** — every operator, every structural keyword, and every built-in function, from the simplest (`+`, `max`) to the most complex (`BARRIER_SURVIVAL`, `CMS`, `TRANCHE_LOSS`). Each entry gives the signature, what it computes, and a short payoff-line example.

It is organised in four parts:

1. **Operators** — the arithmetic / comparison / boolean / ternary layer.
2. **Structural keywords** — `product`, `state`, `events`, `for`, `at`, `receive`, `set`, `exercise`, `knock_in`, `knock_out`, `enter`, `swap`, `bond`, … and the flow flags.
3. **Built-in payoff functions** — ~150 primitives grouped by role, each with a signature and example.
4. **Dates, schedules, and the `for`/`continuous` iterators.**

Throughout, `S`, `SPX`, `asset` denote an underlying name declared in `underlyings = [...];` lowercase identifiers like `perf`, `ki`, `cpn` denote `state` variables or `let` constants.

Part 1 — Operators

Payoff expressions are ordinary infix expressions evaluated **element-wise over Monte-Carlo paths** (every value is a per-path vector). The compiler restricts the expression grammar to a safe whitelist of operators; anything outside it is a compile error. The permitted operators are:

Arithmetic

Operator	Meaning	Example
`+` `-`	add / subtract	`receive S - K`
`*` `/`	multiply / divide	`receive 0.5 * (S1 + S2)`
`//`	floor division	`n_periods // 2`
`%`	modulo	`i % 2`
`**`	power	`receive S ** 2` (power option)
unary `-` `+`	negation / plus	`receive -perf`

Comparison (return a 0/1 mask per path)

Operator	Meaning	Example
`<` `<=`	less / less-or-equal	`perf <= 0.6`

Operator	Meaning	Example
`> >=`	greater / greater-or-equal	`perf >= ab`
`== !=`	equal / not-equal	`idx == 0`

Comparisons produce boolean/`0–1` arrays that you multiply into cashflows or feed to `where`/ternary.

Boolean & bitwise (element-wise path masks)

These combine per-path conditions element-wise (each operand is a 0/1 mask vector over paths):

- `and or not` — scalar boolean operators; use only on scalars, never on per-path arrays (they short-circuit on whole arrays). Example: `not forced`.
- `&` — element-wise AND. Example: `(perf >= cb) & (perf < ab)`.
- `|` — element-wise OR. Example: `(up_breach) | (down_breach)`.
- `^` — element-wise XOR. Example: `a ^ b`.
- `~` — element-wise NOT (bitwise). Example: `~knocked`.

> **Idiom.** Combine per-path conditions with the bitwise `& | ~`, **not** the Python keywords `and/or` — the keywords short-circuit on whole arrays and are only correct for scalars. The autocallable coupon gate is the canonical example: `receive (cpn if (perf >= cb) & (perf < ab) else 0.0)`.

Ternary (the workhorse of payoff logic)

```
<value_if_true> if <condition> else <value_if_false>
```

Evaluated element-wise (compiles to a `where`). Nest freely:

```
receive (1.0 + cpn if perf >= rb
         else (max(perf, floor) if ki > 0.5 else 1.0))
```

`a if cond else b` and `where(cond, a, b)` are interchangeable; the ternary reads better when nested, `where` reads better when `cond` is long.

Indexing, slices, lists, comprehensions

The grammar also allows list/tuple literals, subscripting `x[i]`, slices `x[a:b]`, and **list comprehensions** — used to build per-asset or per-date expressions programmatically inside a factory or a hand-written script:

```
# worst of three explicit ratios, built by comprehension
set worst = min([S/100.0 for S in [SPX, NDX, RTY]])
```

Supported AST node set (whitelist): arithmetic `+ - * / // % **`, unary `- + not ~`, boolean `and or`, bitwise `& | ^`, comparisons `< <= > >= == !=`, the ternary `IfExp`, calls, names, numeric/string constants, lists, tuples, dicts, subscript, slice, and list/generator comprehensions. Attribute access, lambdas, assignments, and imports are **not** permitted inside an expression.

Part 2 — Structural keywords

These are the keywords that give a `.amc` program its shape. They are not functions; they are the statement/block vocabulary.

> **Comments.** A script supports two comment forms. `#` begins a comment that runs to the end of the line (on its own line or trailing after code). `/* … */` is a block comment that may span multiple lines or sit inline between tokens:
>
> ```
> # this whole line is a comment
> let cb = 0.7          # coupon barrier (trailing comment)
> /* a block comment,
>    possibly spanning lines */
> currency = /* inline */ "USD"
> ```
>
> Note `//` is **not** a comment — it is the floor-division operator (`5 // 2`).

Program header

Keyword	Meaning	Example
`product <Name> { … }`	top-level wrapper; `<Name>` is a bare identifier	`product European { … }`
`currency = "USD"`	**mandatory** settlement currency; drives curve discounting off that currency's rate model (registry required)	`currency = "EUR"`
`discount = <rate>`	**override only** — flat short-rate discounting `exp(-r·t)`, no registry; supersedes `currency`-driven discounting when set	`discount = 0.02`
`discount_rate = "USD"`	currency-code form of curve discounting; redundant with `currency` and must match it	`discount_rate = "USD"`
`as_of = "2026-01-01"`	valuation date (ISO) for tenor/date resolution	`as_of = "2026-01-01"`
`underlyings = [SPX, NDX]`	the tradable names referenced in payoffs	`underlyings = ["EURUSD"]`
`param <name> = <v>`	a named compile-time parameter	`param K = 100.0`

> **Discounting: `currency` is the default; don't add a redundant `discount`.** `currency` is **mandatory** and is the canonical way a product is discounted: `currency = "USD"` discounts off that currency's rate-model curve (a wired rate-model registry supplies it). With `currency` set, you do **not** also need `discount` or `discount_rate` — adding them is redundant, and a string `discount_rate` that disagrees with `currency` is a compile error.
>
> Use `discount = <float>` **only** as a deliberate override when you want flat short-rate discounting `exp(-r·t)` with no registry (e.g. a quick stand-alone equity payoff). In that override case the float wins and `currency` becomes a label only (used for FX tagging on cross-currency `pay` events).
>
> Resolution priority, for completeness: a float/callable `discount` wins (then `currency` is a label); a string `discount_rate` is a currency code and must match `currency`; otherwise

> `currency` alone drives curve discounting. So the production default is **just** `currency = "USD"` — leave `discount` out unless you specifically want the flat-rate override.

Declarations

Keyword	Meaning	Example
`let <name> = <const>`	compile-time constant, inlined into expressions	`let cb = 0.7`
`let <name>[K] = [...]`	**static vector** — fixed-size, immutable compile-time list; subscript folds to the element	`let w[3] = [0.2,0.3,0.5]`
`mutable <name> = <const>`	**compile-time counter** — reassignable scalar (not per-path); resolved at lowering	`mutable k = 0`
`mutable <name>[K] = [...]`	**compile-time mutable vector** — for recurrences/tables	`mutable cpn[4] = [0.05,0,0,0]`
`state { … }`	per-path state variables, carried across events	see below
`regressor <name> = <init>`	a state variable usable as an LSM regression feature	`regressor perf = 1.0`
`groups <name> = [A, B, C]`	a named basket the rainbow aggregators operate on	`groups all = [SPX, NDX]`
`func <name>(params) { … }`	a user-defined value function, inlined at each call site	see Functions

Static vectors, `SIZE`, and `mutable` {#kw-vector}. The language has a clean 2×2 of vector forms — (compile-time vs runtime) × (immutable vs mutable):

	immutable	mutable
compile-time	`let v[K] = [...]`	`mutable v[K] = [...]`
runtime, per-path	—	`state { v[K] = init }`

- **`let` vector** — a fixed-size, immutable list of compile-time *values* (not symbols — that's `groups`). The size `[K]` is optional; when given it is asserted against the literal (length mismatch / non-rectangular nested literal is a compile error). Nesting is allowed (`let g[2][3] = [[1,2,3],[4,5,6]]`). Elements may be constant expressions (`let w = [1/3, 1/3, 1/3]`). A subscript `v[i]` with a compile-time index folds to the literal element; a bare `v` used as a list argument (to a basket/aggregator/comprehension) expands to the literal list. It is pure compile-time sugar — it lowers to the inline literals and adds no runtime cost or state.
- **`SIZE` / `LEN` / `LENGTH`** (and `NROWS` / `NCOLS` for nested) — the compile-time length of any compile-time-known iterable: a `let`/`mutable` vector, a literal list, a `groups` name, a `let`-bound `schedule(...)` (its date count), or a `state` array (its declared slot count). It folds to an integer and may be a `range` bound, so `for j in range(0, SIZE(w))` stays length-safe. `SIZE(...)` as a **call** on a `schedule_div(...)` or a runtime per-path value is a compile error — those lengths are market-deferred / per-path and not known at lowering. (Inside a `schedule_div` body, the *bare* token `SIZE` is the engine-substituted dividend count — see the schedule_div section; and `COUNT` is unrelated: the nonzero-count reduction of Part 3.)
- **`mutable` scalar** — a compile-time counter, reassignable with `set k = k + 1`. It is a *single* value (the same for all paths and dates), resolved entirely at lowering, so it is **not** sim-id or time dependent and never reaches the engine. Its assignment RHS must

be compile-time constant (it may use `let`, other mutables, `SIZE`, arithmetic — but not an underlying, `state`, `t`, or any per-path value), and it may be assigned only in compile-time control flow (top level or an unrolled `for`), never under a runtime `if`. For a per-path counter that advances on a data-dependent condition, use `state` + `RUNNING_COUNT`/`RUNNING_SUM` instead.

- `mutable` **vector** — the same, element-wise (`set cpn[k] = cpn[k-1] + 0.0025`). Its niche is **compile-time recurrences** (element `k` depending on `k–1`), which a comprehension cannot express; for independent elements prefer `let` + comprehension. Boundary rule: if an element ever needs a runtime value (`set v[k] = SPX`), it must be `state`, not `mutable`.

```
# weighted basket with a length-safe loop
let w[3] = [0.25, 0.25, 0.5]
groups bk = [SPX, NDX, RTY]
events {
  at 1Y { receive 1000 * sum([w[j] * RATIO(bk[j]) for j in range(0, SIZE(w))]) }
}

# compile-time step-up coupon schedule via a mutable recurrence + counter.
# A plain schedule(start,end,freq) (or dates=[...]) has compile-time-known dates,
# so a mutable inside it UNROLLS at lowering and the counter advances per date.
mutable cpn[4] = [0.05, 0, 0, 0]      # 5% then +25bp each period
mutable k = 1
events {
  for t in schedule(start=1Y, end=4Y, freq=1Y) {
    at t {
      set cpn[k] = cpn[k-1] + 0.0025  # compile-time recurrence
      receive cpn[k-1] * 1000         # this period's coupon (pre-increment)
      set k = k + 1
    }
  }
}
```

> **Mutables and `schedule(...)`.** A compile-time `mutable` advances per date when the loop is **unrolled at lowering**. This happens automatically for a compile-time `for v in [<list>]`/range loop **and** for a `for v in schedule(...)` whose dates are computable at lowering — which is **every plain or calendar/`bda`-rolled `schedule`** (`start/end/freq`, `n=`, or `dates=[...]`), since the calendar roll is static (holiday calendar + `as_of`) and is enumerated via the engine's own date generator, so the unrolled dates match the template path exactly. In those cases the schedule is unrolled (its resolved size grows with the date count) so the mutable can take a different value at each date. The **one** exception is `schedule_div(...)`: its dates are dividend-driven (market-deferred) and not known at lowering, so a compile-time `mutable` there is a **compile error**. But you rarely need one — the engine provides the per-dividend index `current` (the counter) and the dividend count `SIZE` as bare tokens inside a `schedule_div` body, substituted when the dividend dates are known (see the schedule_div section). For a per-path counter use `state` + `RUNNING_COUNT`. A schedule **without** a mutable in its body keeps the compact single-template lowering (resolved size independent of date count) unchanged.
>
> Note schedules include **both endpoints** by default, so `schedule(start=1Y, end=3Y, freq=1Y)` fires at 1Y, 2Y, and 3Y (three dates).

State block. Each entry is `name = init` (optionally `regressor name = init`). A state variable persists between events and is updated with `set`:

```
state {
  regressor perf = 1.0     # used as an LSM regression feature
  ki = 0.0                 # plain carry flag, not a regressor
  memory = 0.0
}
```

> **`baskets` alias.** `baskets <name> = [...]` is an accepted alias for `groups` (singular `basket` / `group` are also accepted). It declares the same named basket that the rainbow aggregators (`WORST_OF`, `BEST_OF_EXCLUDING`, …) operate on.

State variables & regressors

A `state { … }` block declares per-path variables that **persist across events** (carried from one dated event to the next on every path), initialised once at `as_of`. Each entry is `name = <init>`, where `<init>` is a number or an expression. State is the memory of a path-dependent product: running sums, barrier latches, coupon memory, the count of observations so far, and so on. A state variable is read like any other name and updated with `set`:

```
state {
  S_max  = 0.0      # running maximum (lookback)
  memory = 0.0      # accrued unpaid coupon (snowball)
  ki     = 0.0      # knock-in latch (0/1)
}
events {
  for t in schedule(start=6M, end=3Y, freq=S) {
    at t {
      set S_max  = max(S_max, SPX)               # update the running max
      set ki     = max(ki, INDIC(SPX < 60.0))    # latch once, stays 1
      set memory = memory + 0.02                 # accrue
    }
  }
}
```

The `init` may be an expression and is evaluated once; updates inside events are ordinary `set` statements and run in source order. A `set` written **after** a payment in the same event is scheduled so the payment's amount sees the *pre-update* value (so you can pay this period's coupon and then roll the state forward in the same block).

Regressors. Marking a state variable as a **regressor** makes it a feature column in the Longstaff–Schwartz continuation-value regression used by `exercise` / `decide` / `callable` / Bellman `max(CV, …)` events. Choosing good regressors (the state that actually drives the early-exercise boundary — spot, worst-of performance, a rate level, time-to-maturity proxies) is what makes the LSM estimate accurate. Three equivalent spellings are accepted:

```
state {
  regressor perf = 1.0        # preferred: the `regressor` prefix
  perf2 = 1.0 (regressor)     # legacy trailing-flag form (still parses)
  plain = 0.0                 # NOT a regressor — just carried state
}
```

A plain state variable carries its value but is invisible to the regression; a `regressor` does both. For the Bellman `max(CV, …)` form the engine **auto-infers** the regressor, so you often

need not mark anything; mark variables explicitly when you use the `decide { … }` block or want a specific feature set. (You can also pass an explicit `regressors = [a, b]` list on a `decide`; see the optimal-stopping section.)

Array state

State can be **array-valued**, declared `name[K] = init` where **`K` is an integer literal ≥ 1** (a compile-time constant, not an expression). All `K` slots are initialised to the same `init` value. Array state is read and written by index with ordinary subscripts, and the index may itself be a per-path expression:

```
state {
  fixings[6] = 0.0       # 6 fixings, all start at 0
  cnt        = 0.0       # running index into the array
}
events {
  for t in schedule(start=3M, end=18M, freq=Q) {
    at t {
      set fixings[cnt] = SPX     # store this fixing in the current slot
      set cnt = cnt + 1            # advance the index
    }
  }
  at 18M {
    receive AVG([fixings[j] for j in range(0, 6)]) / 100.0
  }
}
```

The decisive property of array state is that the **script size is constant** regardless of `K`: a running counter (or a list comprehension over `range(0, K)`) plus indexed `set` expresses an N-asset or N-date construct (Asian baskets, the Himalaya lock-and-remove mechanism, per-date snapshots) without writing K separate statements. `set name[i] = …` writes one slot; `name[i]` reads one slot; `name` used bare is the whole vector (usable in aggregators via a comprehension such as `AVG([name[j] for j in range(0, K)])`). The index may be a per-path expression. A `regressor` flag may be combined with array state in the same declaration grammar.

User-defined functions

A `func` declares a **reusable value function** that can be called anywhere an expression is allowed (`receive`, `pay`, `set`, `decide`/`exercise` conditions, `knock_out when`, …) and on **any** underlying you pass in. Functions are **pure** (they compute a value; they do not emit events or mutate state) and are **fully inlined** at each call site at compile time — after lowering, no `func` remains in the script, so they are a source-level convenience with zero runtime cost.

A function body is a sequence of `let` bindings, compile-time `for` loops, and `if`/`else` branches, ending in a `return`:

```
func ratio(S, base)        { return S / base - 1.0 }      # one-line body
func capped(S, base, c) {
    let r = ratio(S, base)                          # locals via `let`
    return min(max(r, 0.0), c)                      # any builtin / operator
}
func digital(S, k, pay) {
    if S >= k { return pay } else { return 0.0 }   # if/else
}
func ladder(S, K) {                                 # compile-time fold
    let acc = 0.0
```

```
    for i in range(0, K) { let acc = acc + max(S/100 - 1, 0) }  # carried fold
    return acc / K
}
```

Rules and semantics:

- **Body grammar.** Statements are `let <name> = <expr>`, `for <v> in <iterable> { … }`, `if <cond> { … } else { … }`, and a terminal `return <expr>`. Every path through the body must reach a `return`. Locals are single-assignment (re-`let` to shadow, e.g. the carried accumulator); the full expression vocabulary — all operators, all built-in functions, ternaries, comprehensions, calls to other `func`s — is allowed in every `<expr>`.
- **`if/else` is value selection, not control flow.** `if c { return a } else { return b }` lowers to the vectorised ternary `a if c else b`: **both** branches are evaluated on **every** path and masked, which is the correct (and only) semantics under Monte Carlo. There is no short-circuiting — an early `return` does not skip computation. An `if` without `else` is allowed only when a later `return` covers the fall-through (`if c { return a } return b` ≡ `if c { return a } else { return b }`).
- **Compile-time `for`.** The iterable is a literal list, a `range(...)`, or a group name, and is **unrolled at compile time** into a fold (`acc → g(acc, x₀) → g(that, x₁) → …`). The range/list bound may be a non-literal `let` as long as it **resolves to an integer at compile time** (e.g. `for i in range(0, K)` with `let K = 3`). `return` is not allowed *inside* a loop body — use a ternary for per-iteration conditional values.
- **Composition & defaults.** A `func` may call other `func`s (expanded inside-out; recursion is rejected). Parameters may have defaults (`func capped(S, hi=0.2)`); a trailing default may be omitted at the call site. Arguments are positional.
- **Names.** A `func` may not shadow a built-in function name. Because funcs inline away, they never appear in the canonical/printed script (the expanded expression does).

> **Cost note.** Inlining is by back-substitution, so a local referenced *n* times is substituted *n* times. This is semantically identical (funcs are pure) but can produce large lowered expressions for deeply nested funcs — fine for the engine (it is just an expression), worth knowing for very large fold unrolls.

The event block and iterators

Keyword	Meaning
`events { … }`	the ordered list of dated events
`at <date> { … }`	a single event at one date
`for <var> in <seq> { … }`	**loop**: repeat the block body at every date in `<seq>`
`schedule(start=…, end=…, freq=…)`	generate a periodic date sequence (named keys)
`continuous(start, end, steps=N)`	a fine monitoring grid (for bridge/continuous barriers)

The `for` loop is the single most important structural construct and is worth spelling out, because it is what makes a multi-date product's script *constant size* regardless of the number of observation dates:

```
events {
  for t in [0.5, 1.0, 1.5, 2.0, 2.5, 3.0] {     # explicit date list
    at t {
```

```
      set perf = RATIO(SPX, reset=false)
      skip_last receive (cpn if perf >= cb else 0.0)
      skip_last exercise { forced when perf >= ab; receive 1.0 + cpn }
      only_last receive (1.0 if perf >= rb else max(perf, 0.0))
    }
  }
}
```

The loop variable `t` takes each value of the sequence in turn and is available as the event date. The sequence after `in` may be:

- an **explicit date list** `[0.5, 1.0, …]` (floats, tenors `"6M"`, or ISO dates);
- a **`schedule(start=…, end=…, freq=…)`** call (named keys; e.g. `for t in schedule(start=0, end=5, freq="6M")`);
- a **`continuous(0.0, T, steps=N)`** call — a fine grid for continuous monitoring;
- a **named** sequence bound earlier.

Inside the loop body, the **flow flags** restrict *which iterations* a statement fires on:

Flag	Fires on	Typical use
`skip_last`	every date **except** the final one	intermediate coupons / autocall checks
`only_last`	**only** the final date	maturity redemption
`skip_first`	every date except the first	skip the t_0 fixing
`only_first`	only the first date	initial fixing / premium

Statements inside an event block

Statement	Meaning	Example
`receive <expr>`	pay `<expr>` to the holder (discounted)	`receive max(S - K, 0)`
`pay <expr>`	alias for a negative receive (holder pays); discounted	`pay premium`
`receive_undisc <expr>`	pay `<expr>` to the holder **without discounting**	`receive_undisc 1.0`
`pay_undisc <expr>`	holder pays `<expr>` **without discounting**	`pay_undisc 0.2`
`set <name> = <expr>`	update a state variable	`set perf = S/100.0`
`exercise { … }`	an LSM optimal-stopping / autocall decision (see below)	
`knock_in <name> when <cond>`	latch a knock-in flag the first time `<cond>` is true	`knock_in ki when S <= B`
`knock_out when <cond> [rebate <e>]`	terminate the path when `<cond>`; optional rebate cash	`knock_out when S >= B rebate 0.05`
`enter <inst> when <cond>`	physically enter a declared instrument on exercise	`enter swap when CV < intrinsic`
`cancel <inst> when <cond>`	cancel/terminate an entered instrument	`cancel swap when …`
`pay { … }`	a faithful raw-payment block (advanced)	

Cross-currency (quanto / compo) payments

A cashflow can be paid in a **currency other than the product's** by using the faithful `pay { … }` block form with a `currency` field naming the **foreign cashflow currency**:

```
product CompoNote {
  currency = "USD"          # product (domestic) currency
  discount = 0.02
  underlyings = [SX5E]
  events {
    at 1.0 {
      # EUR-denominated equity payoff, settled into the USD deal:
      pay { amount = max(SX5E - 100.0, 0.0); currency = "EUR" }
    }
  }
}
```

When the `currency` differs from the product currency, the engine **multiplies the amount by the spot FX rate at the event date** to convert it into the product (domestic) currency before discounting and accumulating. The discount curve is **always** the product's domestic curve. This is the mechanism for **compo** (payoff in foreign units, converted at the prevailing FX) cashflows; a genuine **quanto** (foreign payoff converted at a *fixed* FX) is expressed by scaling the amount by a fixed rate in the domestic currency rather than tagging it foreign.

FX resolution is automatic: the engine looks for a registered `FXFactor` whose `domestic_currency` / `foreign_currency` attributes connect the two currencies. If the pairing is ambiguous or you want a specific factor, add an `fx` field naming it explicitly:

```
pay { amount = max(SX5E - 100.0, 0.0); currency = "EUR"; fx = "EURUSD" }
```

This requires a simulation model with the relevant FX factor bound; if none connects the foreign currency to the domestic one, compilation/evaluation raises a clear "no FX factor connecting …" error.

The `currency` and (optional) `fx` fields are accepted on **all four** cashflow keywords — `receive`, `pay`, `receive_undisc`, `pay_undisc` — in either of two forms:

- the **block form** `receive { amount = <expr>; currency = "EUR"; fx = "EURUSD" }` (and likewise for the other three keywords); or
- a **trailing suffix** on the bare-expression statement, `receive <expr> currency "EUR" fx "EURUSD"` (the `fx` clause is optional).

The holder-sign convention is preserved: `receive` / `receive_undisc` pay the (FX-converted) amount **to** the holder, `pay` / `pay_undisc` pay it **from** the holder. The discounted forms (`receive` / `pay`) discount on the domestic curve; the `_undisc` forms do not discount. For FX-converted *performance* (rather than a single converted cashflow) the multi-currency basket builtins (`ABSOLUTE_BASKET` / `RELATIVE_BASKET` with `convert_fx`, §3.4 / §3.14) remain the natural tool.

```
# all four keywords accept the trailing currency suffix:
receive max(SX5E - 100.0, 0.0) currency "EUR"
pay 0.5 currency "EUR" fx "EURUSD"
receive_undisc 1.0 currency "EUR"
pay_undisc 0.2 currency "EUR"
```

`exercise` / `autocall` / `callable` / `putable` / `decide` (optimal stopping)

These all express a **stopping / exercise decision**. At a decision event the engine runs a Longstaff–Schwartz regression to estimate the continuation value and compares it to the immediate payoff. They differ only in who optimises and whether the stop is forced.

`CV` / `CONTINUATION_VALUE` — the continuation value

Inside a decision payoff, the reserved name **`CV`** (long form `CONTINUATION_VALUE`) stands for the **Longstaff–Schwartz continuation value** at the current decision date — the regression estimate of "what the deal is worth if I do *not* stop here." It is one of four reserved names always in scope (no declaration needed): `CV` / `CONTINUATION_VALUE`, plus `EXERCISED` and `HIT` (per-path flags set by the engine's stopping/barrier machinery).

The canonical use is the **Bellman exercise payoff**: a top-level `max` or `min` with `CV` as one of its two arguments. This exact shape is recognised by the resolver and routed to the engine's native `american` event, which performs the LSM regression and auto-infers the regressor — you do not write a `decide` block or list regressors yourself:

- `receive max(CV, <intrinsic>)` — **holder** optimal exercise: stop and take the intrinsic whenever it beats continuing. Lowers to an `american` event.
- `receive min(CV, <intrinsic>)` — **issuer** optimal exercise (callable side): the issuer minimises, calling when continuing is more expensive than the call price.

```
# Bermudan put — holder exercises optimally at each date:
for t in schedule(start=1Y, end=5Y, freq=A) {
  at t { receive max(CV, max(K - SPX, 0.0)) }
}

# Issuer call at par against the continuation value:
at t { receive min(CV, 100.0) }
```

Only the **strict** two-argument `max(CV, X)` / `min(CV, X)` form is the Bellman trigger. A bare `CV`, or `CV + 1`, or `CV` buried deeper in an expression is **not** treated as a Bellman payoff (and the engine rejects a raw `CV` cashflow) — so use the `max`/`min` form, or the explicit `decide { … }` block (below) when you need more control (custom regressors, polarity, an alive window). `CV` may also be referenced inside a `when`/`enter` condition (e.g. `enter S when CV < intrinsic`), where it compares the continuation value against an alternative for a physical switch; multi-date `enter`/`exercise` conditions are required to reference `CV`.

`exercise { … }` — the unified decision statement

Used inside an `at { }` block. Fields (each terminated by `;` or newline; all optional except an immediate payoff source):

- `forced when <cond>` — a **mandatory** stop wherever `<cond>` is true (no optimisation; this is the autocall barrier). Optional.
- `receive <expr>` — the immediate payoff on exercise (the "intrinsic"). This is also what the LSM regression compares against the continuation value.
- `into <instrument>` — **physical settlement** into a declared instrument. *Note: `exercise { into … }` is parsed but not yet lowered by the resolver (it raises "exercise into is not wired yet; use `receive <amount>` for cash settlement"). For physical settlement today, use the `enter <name> when <cond>` statement (below), which is wired.*

- `issuer` — flag: the **issuer** optimises (polarity `min`, i.e. the holder is short the option, as in a callable bond). Absent ⇒ the **holder** optimises (polarity `max`, the default — American / putable / autocall).
- `by <party>` — attribute the decision to `bank` / `counterparty` (used by CCR to know whose option it is).
- `while <cond>` — the decision is only *available* (alive) where `<cond>` holds.

```
# Holder Bermudan put: optimally stop on max(K - S, 0) vs continue
exercise { receive max(100.0 - SPX, 0.0) }

# Autocall: forced redemption at par+coupon when worst-of >= 1.0
exercise { forced when perf >= 1.0; receive 1.0 + cpn }

# Issuer callable bond: the issuer (min) calls at the call price
exercise { issuer; receive 101.0 }

# Physically-settled Bermudan swaption: enter the swap when optimal
# (exercise { into S } is not yet wired — use `enter` for now)
enter S when CV < intrinsic
```

`autocall { … }` — forced (non-optimised) stop

Sugar for a purely **forced** stop with no continuation regression — the path terminates wherever the trigger condition holds, paying the value. **Both** fields are required: `when = <cond>` (the autocall trigger) and `pay = <amount>` (paid on the stop):

```
autocall { when = perf >= 1.0; pay = 1.0 }
```

In practice the `exercise { forced when …; receive … }` form above is an equivalent alternative; `autocall` is the compact block form.

`callable { … }` / `putable { … }` — priced against a call/put price

Desugar to a `decide` with the issuer (`callable`, polarity `min`) or holder (`putable`, polarity `max`) optimising against a price given by `pay`. The required field is `pay = <amount>` (the call/put price); an optional `when = <cond>` restricts the eligibility window:

```
callable { pay = 100.0 }              # issuer may call at par (optimal)
callable { pay = 101.0; when = t >= 1.0 } # callable only from year 1
putable  { pay = 99.0 }                   # holder may put at 99
```

`decide { … }` — the explicit low-level form

The primitive all of the above desugar to. Fields (as `name = value`):

- `immediate = <expr>` — payoff if the path stops here.
- `alive = <cond>` — where the decision is available (default `True`).
- `forced = <cond>` (alias `forced_stop`) — where stopping is mandatory.
- `regressors = [name, …]` — the state variables used as continuation-value regression features.
- `polarity = max | min` — holder optimises (`max`, default) or issuer (`min`).
- `controls = [(label, expr), …]` — extra regression basis columns (advanced).

```
decide {
  immediate  = conversion_ratio * SPX
  alive      = alive_now > 0
  regressors = [SPX]
  polarity   = max
}
```

`convertible { … }` is the same machinery specialised to the holder's convert-vs-continue choice in a convertible bond.

Instrument declarations (physical settlement)

A product that exercises *into* a tradable instrument declares it at the top of the `product` block and references it from an `exercise … into <name>` (or `enter <name> when <cond>`) statement. The engine then values the entered instrument from the exercise date forward. Three instrument kinds exist: `swap`, `bond`, `forward`.

`swap <name> { … }`

A vanilla or multi-leg interest-rate swap. The body is a mix of `key = value` settings and explicit `pay|receive <leg-type> { … }` leg blocks.

Single-leg shorthand settings (a plain vanilla fixed/float swap):

- `maturity = <date>` — swap held from inception to this date; **or**
- `tenor = <length>` — for a swaption's underlying, length anchored to the exercise date.
- `fixed_rate = <rate>` — the fixed coupon (e.g. `3%` or `0.03`).
- `frequency = <freq>` — payment frequency (`6M`, `1Y`, …). Default `6M`.
- `pay = fixed | float` — which leg the holder **pays** (`fixed` ⇒ payer swap).
- `float_spread = <rate>` — spread over the float index.
- `float = LIBOR | CMS | OIS` — the float index type (default LIBOR-style).
- `cms_tenor = <length>`, `cms_freq = <freq>` — for a CMS-float swap, the constant-maturity underlying-swap tenor and its frequency.
- `notional = <amount>` (default 1.0), `currency = "<ccy>"`.

```
swap S {
  maturity   = 10.0
  fixed_rate = 3%
  frequency  = 6M
  pay        = fixed          # payer swap: pay fixed, receive float
  float      = LIBOR
}
```

Multi-leg form — one or more explicit `pay|receive <type> { … }` leg blocks. Leg `<type>` is one of `fixed`, `float`, `cms`, `cms_spread`. Each leg block takes keys:

- common: `frequency = <freq>` (default `S` = semi-annual), `notional = <amt>`, `currency = "<ccy>"`, `exchange_notional = true|false` (exchange principal at start/end — cross-currency swaps).
- `fixed` leg: `rate = <rate>`.
- `float` leg: `index = <ccy/curve>` (alias `forward`), `spread = <rate>`, `fixing = in_advance | in_arrears` (compounding convention; `in_arrears` ⇒ compounded RFR).

- `cms` leg: `swap_tenor = <length>` (the constant-maturity tenor), `index`, `spread`, optional `swap_freq` and `swap_day_count`.
- `cms_spread` leg: `swap_tenor` and `swap_tenor2` (the two CMS tenors), `spread`.

```
# Cross-currency basis swap: USD float vs EUR float w/ notionals
swap XCCY {
  pay    float { index = "USD"; frequency = 3M; spread = 0.0;
                 exchange_notional = true }
  receive float { index = "EUR"; frequency = 3M; spread = 0.0025;
                 exchange_notional = true }
}

# CMS-spread leg vs fixed:
swap STEEPENER {
  receive cms_spread { swap_tenor = 10.0; swap_tenor2 = 2.0;
                       spread = 0.0; frequency = 1Y }
  pay     fixed       { rate = 0.01; frequency = 1Y }
}

# OIS leg (compounded in arrears):
swap OIS_SWAP {
  receive float { index = "USD"; fixing = in_arrears; frequency = 1Y }
  pay     fixed { rate = 0.025; frequency = 1Y }
}
```

`bond <name> { … }`

A fixed-coupon bond, optionally **defaultable**. Keys:

- `maturity = <date>` — redemption date (required).
- `coupon = <rate>` — coupon rate (required; alias `fixed_rate`).
- `frequency = <freq>` — coupon frequency (default `1Y`).
- `notional = <amount>` (default 100.0); `face_value` is also accepted for the redemption amount.
- `issuer = "<name>"` — makes it a **defaultable** bond: the engine survival-weights the coupons and integrates recovery at the default time.
- `recovery = <fraction>` — recovery rate on default (else taken from the issuer's curve).

```
# Risk-free coupon bond
bond B { maturity = 5.0; coupon = 4%; frequency = 1Y; notional = 100 }

# Defaultable bond (survival-weighted coupons + recovery on default)
bond RB { maturity = 5.0; coupon = 5%; issuer = "ACME"; recovery = 0.4 }
```

`forward <name> { … }`

An equity or FX forward to settle into. Keys:

- `underlying = <asset>` — equity forward (settles into the asset); **or**
- `pair = "<CCY1CCY2>"` — FX forward (settles into a physical FX forward; `enter`-ing it lowers to `fx_physical_exercise`).
- `strike = <level>` — the forward strike (required).
- `maturity = <date>` — settlement date (required).
- `long = true|false` — long or short the forward (default `true`).
- `notional = <amount>` — optional.

```
forward F  { underlying = SPX; strike = 100.0; maturity = 1.0; long = true }
forward FX { pair = "EURUSD"; strike = 1.10; maturity = 0.5 }
```

`convertible <name> { … }` — convertible-bond instrument

A convertible bond is declared as a top-level instrument with flat settings plus optional nested blocks. The complete set of flat settings (any other key is a compile error) is: `underlying`, `notional`, `maturity`, `coupon` (percent literal allowed, e.g. `2%`), `frequency`, `ratio` (shares per bond), `currency`, `issuer` and `recovery` (for a defaultable issuer), `start`, `settlement`, and `mandatory_conversion`. The nested blocks are `convert`, `call`, `put`, `reset`, and `change_of_control`; each holds `key = value` fields plus, where applicable, the trigger clauses `contingent …`, `soft …`, and `hard …`:

- `convert { window = [<from>, <to>]; contingent when <cond> }` — the conversion right over a date window, optionally contingent on a path condition. Inside a convertible's clauses, `conv_price` names the conversion price.
- `call { schedule = [...]; price = <pct>; soft when <cond> for N of last M days; hard from <date>; make_whole = true|false }` — the issuer call, with optional soft-call (conditional) and hard-call (from a date) provisions and a make-whole flag.
- `put { dates = [...]; price = <pct> }` — the holder put.
- `reset { dates = [...]; floor = <expr> }` — conversion-price reset with a floor.

```
convertible C {
  notional = 1000; maturity = 5Y; coupon = 2%; frequency = A
  underlying = ACME_EQ; ratio = 20.0
  issuer = "ACME"; recovery = 0.40
  convert { window = [1Y, 5Y]; contingent when ACME_EQ >= 1.30 * conv_price }
  call    { schedule = [2Y, 3Y, 4Y]; price = 102%
            soft when ACME_EQ >= 1.30 * conv_price for 20 of last 30 days
            hard from 4Y; make_whole = true }
  put     { dates = [3Y]; price = 100% }
  reset   { dates = [2Y, 4Y]; floor = 0.80 * conv_price }
}
```

(This is the instrument-*declaration* form; the in-events `convertible { … }` keyword in the exercise section is the per-event holder decision.)

A convertible **self-expands**: declaring it generates its full event stream (coupons, redemption, and the windowed convert/call/put/reset decisions) automatically, so an `events { … }` block is not required. By default the bond is issued at `t0`; its issue date can be deferred in two equivalent ways:

- the `start` setting — `start = 1Y` issues the bond 1 year forward; or
- an `at <date> enter <name>` statement in the events block — `at 1Y enter C` issues convertible `C` at 1Y.

Both shift the *entire* timeline (coupons, redemption, and every `window` / `schedule` / `dates` inside the block) relative to the issue date, and are exactly equivalent (identical events, identical price); an `at … enter` overrides a `start =` setting if both are given. Unlike `swap` / `bond` / `forward`, `enter`-ing a convertible is **not** a physical-delivery decision (a convertible already contains its own optionality) — it only sets the issue date. `cancel` is not supported for a convertible; use a `put` / `call` clause inside the block instead.

```
# Deferred-issue convertible — issue date set in events (1Y fwd):
product DeferredCB {
  currency = "USD"; discount = 0.03
  convertible C {
    notional = 1000; maturity = 4Y; coupon = 3%; frequency = A
    underlying = ACME_EQ; ratio = 25.0
    convert { window = [1Y, 4Y]; contingent when ACME_EQ >= 1.30 * conv_price }
    call    { schedule = [2Y, 3Y]; price = 102%; hard from 3Y }
    put     { dates = [2Y]; price = 100% }
  }
  events { at 1Y enter C }      # same as `start = 1Y` on convertible
}
```

Entering an instrument

```
# Enter the declared swap S when a per-path condition holds (wired):
enter S when CV < intrinsic;
# Cancel a previously-entered instrument:
cancel S when called;
# Alternatively, attribute the option to a party with a by-block
# (mutually exclusive with the `when` clause):
enter S { by bank }
cancel S { by counterparty }
```

> **State of physical settlement.** Instrument *declarations* (`swap`/`bond`/ `forward`) and the `enter`/`cancel` statements are wired. The `exercise { into <name> }` form is parsed but not yet lowered — the resolver raises a clear error directing you to cash settlement (`receive <amount>`) or the `enter` statement. Reimplementations should accept the `into` token in the grammar but may defer its lowering.

Part 3 — Built-in payoff functions

Every function below can be called inside a `receive` / `set` / `exercise` expression. All are **element-wise over Monte-Carlo paths**.

> **Case-insensitivity.** Built-in **function names are case-insensitive**: `max`, `MAX`, `Max`, and `mAx` all resolve to the same function, as do `running_mean`, `RUNNING_MEAN`, and `Running_Mean`. The resolver canonicalises any spelling to the function's upper-case form (which the engine always provides) before lowering. This applies to *all* built-in functions — arithmetic, aggregators, accumulators, rates, credit, everything in this Part.
>
> Two things that are **not** case-folded: (1) **structural keywords** (`product`, `state`, `events`, `at`, `for`, `receive`, `exercise`, the flow flags, …) — these are fixed lower-case tokens and must be written exactly; and (2) **identifiers you declare** — underlying names, `state` variables, `let` constants, `groups` names, and string labels (e.g. the first argument of `RUNNING_*` / `SNAPSHOT` / `BARRIER_SURVIVAL`) — these are ordinary names and are matched exactly as written. Many functions also have **aliases** (e.g. `avg` = `AVERAGE` = `MEAN`); each entry lists them.

Each entry gives the **full signature** (including every keyword argument and its default) and a concrete **example** payoff line. Functions are positional-only unless a keyword is shown with `=`; the kwarg-bearing families are exactly those listed here with `kw=default`.

3.1 Arithmetic & shaping

Element-wise scalar functions. All accept and return per-path arrays.

- `max(a, b, …)` / `min(a, b, …)` — element-wise maximum / minimum of all arguments. `receive max(SPX - K, 0.0)`
- `abs(x)` — absolute value. `receive abs(S1 - S2)`
- `sqrt(x)` — square root. `set v = sqrt(realized)`
- `exp(x)` / `log(x)` — e^x and natural log (`log` alias `LN`). `set df = exp(-r * t)` · `set lr = log(SPX / s_prev)`
- `log10(x)`, `log2(x)`, `log1p(x)` (= ln(1+x)), `expm1(x)` (= e^x-1).
- `pow(x, p)` — x^p, identical to the `**` operator. `receive pow(SPX / s0, 2)`
- `sign(x)` — −1 / 0 / +1. `set d = sign(SPX - K)`
- `pos(x)` / `neg(x)` — positive part `max(x,0)` / negative part `max(-x,0)` (aliases `positive_part` / `negative_part`). `receive pos(SPX - K)`
- `ceil(x)` / `floor(x)` — round up / down. `set n = floor(perf)`
- `clamp(x, lo, hi)` — constrain to `[lo, hi]` (aliases `clip`, `CLAMP`, `CLIP`). `set p = clamp(perf, 0.0, 1.5)`
- `cap_at(x, c)` — `min(x, c)` (aliases `capped_at`, `CAP_AT`). `receive cap_at(coupon, 0.08)`
- `floor_at(x, f)` — `max(x, f)` (aliases `floored_at`, `FLOOR_AT`). `receive floor_at(perf, 0.0)`

3.2 Payoff & option shapes

- `call_payoff(S, K)` — `max(S − K, 0)` (alias `CALL`). `receive call_payoff(SPX, 100.0)`
- `put_payoff(S, K)` — `max(K − S, 0)` (alias `PUT`). `receive put_payoff(SPX, 100.0)`
- `call_spread(S, K1, K2)` — long-`K1` / short-`K2` call spread. `receive call_spread(SPX, 100, 120)`
- `straddle(S, K)` — `abs(S − K)`. `receive straddle(SPX, 100.0)`
- `collar(S, lo, hi)` — floor the payoff at `lo`, cap at `hi`. `receive collar(perf, 0.9, 1.1)`
- `digital(cond)` — 1 where `cond` else 0 (aliases `DIGITAL`, `INDICATOR`, `indic`, `indicator_of`). `receive 0.10 * digital(SPX >= 110)`
- `double_digital(S, lo, hi)` — 1 inside the band `[lo, hi]`, else 0. `receive double_digital(SPX, 95, 105)`
- `range_digital(S, lo, hi)` — corridor indicator (synonym of `double_digital`).
- `ramp(x, lo, hi)` — 0 below `lo`, linear up to 1 at `hi`, flat 1 above. `receive ramp(perf, 0.8, 1.0)`
- `ladder(x, [levels])` — locked-step payoff over the ascending `levels`. `receive ladder(perf, [1.1, 1.2, 1.3])`
- `step(x)` — Heaviside (1 if `x ≥ 0`) (alias `HEAVISIDE`). `receive step(SPX - K)`

3.3 Smoothed / differentiable shapes

Discontinuous payoffs (digitals, barriers, steps) make Monte-Carlo Greeks noisy and destabilise the LSM continuation-value regression. The smoothed family replaces a hard step with a differentiable transition of controllable width. Each function is documented below with its full signature, the mathematical formula it computes, every keyword, and an example.

`sigmoid(x, scale=1.0)`

The logistic function — a smooth 0→1 transition centred at `x = 0`:

$$\text{sigmoid}(x,\ \text{scale}) = \frac{1}{1 + e^{-x/\text{scale}}}$$

`scale` controls the transition width: **small** `scale` approaches a hard `step(x)`; **large** `scale` is gentle. (Note the sign: the argument is `-x/scale`, so increasing `scale` *widens* the transition.)

```
receive sigmoid(SPX - B)            # transition centred at barrier B
receive sigmoid(SPX - B, scale=0.5)     # wider, smoother transition
```

`smooth_step(x, K, smoothing=None, kind=None, reference=1.0)` and `smooth_indicator(...)`

A differentiable replacement for `step(x - K)` / `digital(x >= K)`: a value rising smoothly from 0 to 1 as `x` crosses the level `K`. `smooth_indicator` is the same function under a second name. The transition half-width is

> `δ = smoothing · reference`

(so `smoothing` is a *fraction of* `reference`, typically the inception spot `S0`). The `kind` keyword selects one of three transition shapes:

- `"call_spread"` (default) — a piecewise-linear ramp over `[K−δ, K+δ]`:

  $$f(x) = \text{clip}\left(\frac{x - (K - \delta)}{2\delta},\ 0,\ 1\right), \quad \delta = \text{smoothing} \cdot \text{reference}$$

- `"logistic"` — a logistic transition of width `δ`:

  $$f(x) = \frac{1}{1 + e^{-(x-K)/\delta}}$$

- `"tanh"` — a hyperbolic-tangent transition:

  $$f(x) = \tfrac{1}{2}\left(1 + \tanh\tfrac{x - K}{\delta}\right)$$

Arguments: `x` (the observable), `K` (the threshold), `smoothing` (the half-width as a fraction of `reference`; `None` ⇒ a library default of ≈1% of `reference`), `kind` (the shape, default `"call_spread"`), `reference` (the scale, default `1.0`).

```
# Knock-out indicator at S = 80, smoothed over δ = 1% · 100 = 1.0:
receive smooth_step(SPX, 80, 0.01, reference=100)

# Logistic digital coupon at the 70% performance barrier:
receive 0.06 * smooth_step(perf, 0.70, 0.02, kind="logistic")
```

`call_spread(x, K, smoothing=None, reference=1.0)`

Shorthand for `smooth_step(x, K, smoothing, "call_spread", reference)` — the piecewise-linear smoothed step. Same `smoothing`/`reference` semantics.

```
receive call_spread(SPX, 100, 0.01, reference=100)
```

`softmax(beta, a, b, …)` and `softmin(beta, a, b, …)`

Smooth, differentiable approximations of `max` / `min`. **The first argument is the sharpness `beta`**, not a value:

$$\text{softmax}(\beta,\, x_1,\, \ldots,\, x_n) = \frac{1}{\beta} \log \sum_{i=1}^{n} e^{\beta x_i}$$

$$\text{softmin}(\beta,\, x_1,\, \ldots,\, x_n) = -\,\text{softmax}(\beta,\, -x_1,\, \ldots,\, -x_n)$$

As `β → +∞`, `softmax` approaches the hard `max`; as `β → 0` it approaches the average. `softmin` is `softmax` with negated inputs (equivalently, pass a negative `β` to `softmax`).

```
set wp = softmin(20, perf1, perf2)    # smooth worst-of, sharpness 20
set bp = softmax(20, S1, S2, S3)          # smooth best-of
set wp = softmax(-20, perf1, perf2)   # smooth min via β<0
```

> **Aliases.** `SOFT_MAX` / `SOFTMAX` / `soft_max` and `SOFT_MIN` / `SOFTMIN` / `soft_min` are all accepted, as are `SMOOTH_STEP` / `SMOOTH_INDICATOR`, `CALL_SPREAD`, and `SIGMOID`. (And, like all functions, they are case-insensitive — see the note at the start of Part 3.)

3.4 Basket / rainbow aggregators

These operate across the members of a basket. A basket is supplied as a `groups`-declared name, a comma-separated argument list, or a list literal. When applied to a vectorised expression like `RATIO(all, reset=false)` the resolver distributes it over the group members, so the compiled product is identical to the explicit per-asset form.

Order statistics and selection.

- `WORST_OF(group)` (aliases `WORST`, `worst_of`) — the smallest member. `set wp = WORST_OF(RATIO(all, reset=false))`
- `BEST_OF(group)` (aliases `BEST`, `best_of`) — the largest member. `set bp = BEST_OF(RATIO(all, reset=false))`
- `KTH_BEST(group, k)` / `KTH_WORST(group, k)` — the k-th largest / smallest (1-based). `set second = KTH_BEST(all, 2)`
- `NTH_BEST(group, n)` / `NTH_WORST(group, n)` — synonyms of the above.
- `MEDIAN(group)` — the median member. `set m = MEDIAN(all)`
- `PERCENTILE(group, q)` / `QUANTILE(group, q)` — the q-quantile (`q ∈ [0,1]`). `set p90 = PERCENTILE(all, 0.9)`

Index returners (which member, not its value).

- `ARGMAX_OF(group)` / `ARGMIN_OF(group)` — index of the best / worst member.
- `WORST_OF_IDX(group)` / `BEST_OF_IDX(group)` — same, explicit names. `set i = BEST_OF_IDX(all)`

Sums and means.

- `SUM(a, b, …)` — sum of the arguments. `receive SUM(leg1, leg2)`

- `PROD(a, b, …)` / `PRODUCT(a, b, …)` — product. `set g = PROD(gross_rets)`
- `SUM_LOG(a, b, …)` — Σ log (geometric-basket building block).
- `AVERAGE(group)` (aliases `MEAN`, `AVG`, `avg`) — arithmetic mean. `set m = AVG(S1, S2, S3)`
- `SUM_BEST_K(group, k)` / `SUM_WORST_K(group, k)` (aliases `SUM_BEST`, `SUM_WORST`) — sum of the best / worst `k`. `receive SUM_BEST_K(all, 3)`
- `BASKET(weights, assets)` — weighted basket $\Sigma\, w_i \cdot \text{asset}_i$. Both arguments are lists of equal length. `set b = BASKET([0.5, 0.5], [SPX, NDX])`

Dispersion / ranking.

- `VAR(group)` / `VARIANCE(group)` — population variance across members.
- `STDEV(group)` (aliases `STD`, `STDDEV`) — standard deviation.
- `RANGE(group)` — `max − min` (basket spread). `set s = RANGE(all)`
- `RMS(group)` — root-mean-square.
- `RANK_OF(group, member)` (alias `RANK`) — ordinal rank of a named member.
- `PCT_RANK(group, x)` (aliases `PERCENTILE_RANK`, `PCTRANK`) — percentile rank of `x` within the basket.
- `COUNT([c1, c2, …])`, `ANY([…])`, `ALL([…])` — count / any / all of a list of boolean masks. `set n = COUNT([a, b, c])`

Exclusion-aware aggregators (Himalaya / Altiplano lock-and-remove).

Each takes the basket plus an `exclude` argument — an index array (typically the `excl[]` state array) marking members already "locked out" on earlier dates. They all accept a `min_keep=0` keyword: the minimum number of members that must remain active when too many have been excluded (graceful degradation), defaulting to 0.

- `BEST_OF_EXCLUDING(group, exclude, min_keep=0)` / `WORST_OF_EXCLUDING(group, exclude, min_keep=0)` — best / worst over the members not in `exclude`. `set b = BEST_OF_EXCLUDING(RATIO(all, reset=false), excl)`
- `BEST_OF_IDX_EXCLUDING(group, exclude, min_keep=0)` / `WORST_OF_IDX_EXCLUDING(group, exclude, min_keep=0)` — the *index* of the best / worst surviving member (this is the value you append to `excl` to lock it). `set j = BEST_OF_IDX_EXCLUDING(perf_all, excl)`
- `EXCLUDE_CONT_BEST(group, exclude, min_keep=0)` / `EXCLUDE_CONT_WORST(group, exclude, min_keep=0)` — continuous-relaxation versions (smoothed lock-and-remove) for differentiable Himalaya Greeks.
- `AVG_EXCLUDING(group, exclude, min_keep=0)`, `SUM_EXCLUDING(...)`, `MEDIAN_EXCLUDING(...)`, `STDEV_EXCLUDING(...)` (aliases `STD_EXCLUDING`, `STDDEV_EXCLUDING`), `VAR_EXCLUDING(...)` (alias `VARIANCE_EXCLUDING`) — aggregate over the un-locked members only. `set m = AVG_EXCLUDING(RATIO(all, reset=false), excl)`

A worked Himalaya step (best performer locked and removed each date):

```
state {
  excl[3] = -1.0       # array state: indices locked so far (-1 = none)
  basket_sum = 0.0
}
events {
  for t in [1.0, 2.0, 3.0] {
    at t {
      set best_idx = BEST_OF_IDX_EXCLUDING(RATIO(all, reset=false), excl)
      set basket_sum = basket_sum
                       + BEST_OF_EXCLUDING(RATIO(all, reset=false), excl)
      set excl[t] = best_idx        # lock this date's winner out
    }
  }
```

```
  at 3.0 { only_last receive basket_sum / 3.0 }
}
```

3.5 Path-performance (ratio from a reference fixing)

The `RATIO` / `PERF` family converts spot to a performance relative to a stored reference fixing, with optional running cap/floor. The reference is captured on the first call per path. **Keyword arguments:** `reset` (bool), `K` (strike offset), `cap`, `floor`.

- `RATIO(asset, reset=false, cap=None, floor=None)` — spot ÷ reference fixing. `reset=false` keeps the inception fixing (cumulative, autocallable-style); `reset=true` re-bases the reference to the current spot each call (local, cliquet-style). `cap`/`floor` bound the returned ratio. `set perf = RATIO(SPX, reset=false)`
- `PERF(asset, type="relative", K=1.0, cap=None, floor=None)` — the return form (`RATIO − 1` when `K=1`). `type` selects the performance definition (`"relative"` is the ratio form). `set r = PERF(SPX, cap=0.1, floor=-0.1)`
- `RATIO_SUM(asset, reset=false, cap=None, floor=None)` / `RATIO_PROD(asset, …)` — cumulative sum / product of the per-call ratios.
- `PERF_SUM(asset, K=1.0, cap=None, floor=None)` / `PERF_PROD(asset, …)` — cumulative capped/floored returns (cliquet accumulation). `set c = PERF_SUM(SPX, cap=0.05, floor=-0.05)`
- `RATIO_SUM_IF(asset, cond, …)`, `PERF_SUM_IF(asset, cond, …)`, `RATIO_PROD_IF`, `PERF_PROD_IF` — accumulate only on calls where `cond` is true (regime-gated). `set c = PERF_SUM_IF(SPX, SPX > 90, cap=0.05)`

> **"Get the value of an underlying at a time" — there is no `SPOT(asset, t)`.** A bare underlying name *is* its simulated value at the current event's date, so read a level by placing the event there, not by passing a date. To use a level *elsewhere*, capture it with `SNAPSHOT(label, asset)` in the `at t` block and reference the label later; for a *historical observed* fixing use `PAST_FIX(name, when)` / `FIXING(name, when)`. "Performance since inception" is already `RATIO(asset, reset=false)`.

3.6 Running accumulators (self-seeding per-path state)

Each maintains hidden per-path state keyed by a **string label** (first argument), so a single call inside a `for` loop accumulates across all iterations. **Keyword argument:** `init` (the seed value). The `_IF` variants take the **condition as the second argument**, then the value.

- `RUNNING_MIN(label, value, init=None)` — running minimum. Alias `INF`. The idiomatic discrete knock-in tracker: `set ki = INDIC(INF("wmin", perf) < 0.6)`
- `RUNNING_MAX(label, value, init=None)` — running maximum. Alias `SUP`. `set hi = SUP("hi", SPX)`
- `RUNNING_SUM(label, value, init=0.0)` — running sum. `set tot = RUNNING_SUM("acc", cpn)`
- `RUNNING_MEAN(label, value, init=0.0)` — running mean (arithmetic Asian average). Alias `RUNNING_AVG`. `set avg = RUNNING_MEAN("avg", SPX)`
- `RUNNING_PROD(label, value, init=1.0)` — running product. Alias `RUNNING_PRODUCT`. `set comp = RUNNING_PROD("c", 1 + r)`
- `RUNNING_SUM_LOG(label, value, init=0.0)` — running Σ log (geometric Asian).

- `RUNNING_COUNT(label, init=0.0)` — count of events seen (no value argument). `set n = RUNNING_COUNT("n")`
- `RUNNING_SUM_IF(label, cond, value, init=0.0)` — accumulate the value only on calls where `cond` is true. Same shape for `RUNNING_PROD_IF`, `RUNNING_MEAN_IF`, `RUNNING_SUM_LOG_IF`. `set s = RUNNING_SUM_IF("s", SPX > 90, cpn)`
- `RUNNING_COUNT_IF(label, cond, init=0.0)` — count of calls where `cond` holds (range-accrual day counter). `set d = RUNNING_COUNT_IF("d", (SPX > 95) & (SPX < 105))`

3.7 Barrier / corridor survival (continuous Brownian-bridge monitoring)

Per-path probability the barrier was **not** breached between observation dates, computed via the Brownian-bridge no-hit factor. The function self-manages the previous spot and time across calls (so one call per fine-grid step accumulates the survival product). The terminal knock-in flag is `1 - surv`.

`BARRIER_SURVIVAL(label, value, barrier, direction, monitoring="discrete", sigma=None, smoothing=None, kind=None, reference=1.0, init=1.0, var_mode="cross_path", var_halflife=None)`

- `label` — string state key.
- `value` — the monitored observable (spot / ratio).
- `barrier` — the barrier level.
- `direction` — `"down"` (knock when `value` falls through `barrier`) or `"up"`.
- `monitoring` — `"discrete"` (check only at call dates) or `"continuous"` (Brownian-bridge between calls).
- `sigma` — per-path volatility used by the bridge; if `None`, auto-sourced from the model / path dispersion per `var_mode`.
- `smoothing`, `kind`, `reference` — smoothing of the breach indicator (as in `smooth_step`); `reference` is the scale.
- `init` — survival-probability seed (default 1.0).
- `var_mode` — how `sigma` is estimated when not given (`"cross_path"` default).
- `var_halflife` — EWMA half-life for the variance estimate, if applicable.

```
set surv = BARRIER_SURVIVAL("ki", SPX, 60.0, "down",
                            monitoring="continuous", sigma=0.25)
only_last set ki = 1.0 - surv
```

`CORRIDOR_SURVIVAL(label, value, lower, upper, monitoring="discrete", sigma=None, var_mode="cross_path", var_halflife=None, …)`

Two-sided version: survival probability of staying inside the corridor `[lower, upper]`. Same monitoring/`sigma` semantics.

```
set surv = CORRIDOR_SURVIVAL("c", SPX, 90, 110,
                             monitoring="continuous", sigma=0.2)
```

3.8 Realised-moment accumulators (variance / vol products)

Running realised moments of log-returns. **Keyword argument:** `annualization` (trading-day factor, default 252). The `_IF` variants gate on a condition.

- `REALIZED_VAR(label, value, annualization=252)` — running realised variance. Aliases `REALISED_VAR`, `REALIZED_VARIANCE`. `set rv = REALIZED_VAR("rv", SPX)`
- `REALIZED_VOL(label, value, annualization=252)` — √ realised variance. `set vol = REALIZED_VOL("v", SPX)`
- `REALIZED_QV(label, value, annualization=252)` — quadratic variation.
- `REALIZED_QV_CROSS(label, value_x, value_y, annualization=252)` — cross quadratic variation (correlation swaps). `set c = REALIZED_QV_CROSS("c", SPX, NDX)`
- `REALIZED_VAR_IF(label, value, cond, annualization=252)` (and `_VOL_IF`, `_QV_IF`) — regime-gated realised moment. `set rv = REALIZED_VAR_IF("rv", SPX, SPX > 90)`

3.9 Rates functions

- `LIBOR(currency, t_start, t_end, tenor=None)` — forward IBOR fixing over `[t_start, t_end]`. `tenor` names the index when it differs from `t_end − t_start` (defaults to that span with a 14-day snap tolerance). `receive tau * LIBOR("USD", t, t + 0.5)`
- `CMS(currency, t_start, n_payments, freq=1.0, tenor=None)` — constant-maturity swap rate of an underlying swap with `n_payments` periods of length `freq` years. `receive tau * CMS("USD", t, 10, 1.0)`
- `OIS(currency, t_start, t_end)` — OIS rate over the period. `receive tau * OIS("USD", t, t + 0.25)`
- `RFR(currency, t_start, t_end, index=None)` — compounded risk-free rate in arrears; `index` overrides the per-currency canonical RFR. `receive tau * RFR("USD", t, t + 0.25)`
- `DF(currency, t_pay)` — discount factor from now to `t_pay`. `receive DF("USD", 3.0) * 1.0`
- `FORWARD(asset, t)` — forward price/level at `t`. Alias `FWD`. `set f = FORWARD(SPX, 1.0)`
- `SWAP(...)` — par swap rate / swap PV. Accepts the leg-dict form `SWAP(legs=[{...}, ...])` or the vanilla shortcut `SWAP(maturity=…, frequency=…, fixed_rate=…, side=…, notional=…, discount=…, spread=…)`. `receive SWAP(maturity=5, frequency="6M", fixed_rate=0.03)`
- `INTERP(x, xs, ys)` — linear interpolation of `ys` over knots `xs` at `x`. Aliases `LINTERP`, `LERP`. `set r = INTERP(t, [0, 1], [0.02, 0.03])`

3.10 Credit functions

- `IS_ALIVE(issuer)` — 1 while `issuer` has not defaulted, 0 after. `receive IS_ALIVE("ACME") * cpn`
- `RECOVERY(issuer)` — recovery rate realised at the default time. `receive just_def * RECOVERY("ACME") * face`
- `HAZARD(issuer)` — instantaneous default intensity. `set h = HAZARD("ACME")`
- `JUST_DEFAULTED(issuer, alive_prev)` — 1 on the step `issuer` defaults (given the previous-step survival indicator). `set jd = JUST_DEFAULTED("ACME", alive_prev)`
- `BOND(coupon_rate=…, maturity=…, frequency=…, notional=1.0, issuer_name=…, recovery_rate=…)` — survival-weighted defaultable-bond PV at the current event date (all keyword arguments; exactly one of `coupon_rate` for a fixed bond). Recovery is integrated at the default time. `receive BOND(coupon_rate=0.04, maturity=5, issuer_name="ACME")`

- `TRANCHE_LOSS(names, attach, detach, weights=None, notional=1.0)` — loss in the tranche slice `[attach, detach]` of the weighted portfolio, as a fraction of the tranche width ($\in [0,1]$ per path). `set L = TRANCHE_LOSS(book, 0.03, 0.07)`
- `INDEX_SPREAD_NOTIONAL(names, weights=None)` — outstanding (un-defaulted) notional of a credit index. `set n = INDEX_SPREAD_NOTIONAL(book)`

3.11 Inflation & dividends

- `YOY_INFLATION(index, t_start, t_end, nominal=…, real=…)` — year-on-year inflation rate of the CPI `index`. `nominal`/`real` override the registry currency hints from the CPI factor. `receive tau * YOY_INFLATION("CPI", t, t + 1)`
- `ZC_INFLATION_SWAP_RATE(index, t_start, t_end, nominal=…, real=…)` — zero-coupon breakeven rate. `set br = ZC_INFLATION_SWAP_RATE("CPI", 0, 5)`
- The dividend macros `DIV_AMOUNT`, `FWD_DIV_AMOUNT`, `DIV_C`, `DIV_Y`, `DIV_T_EX`, `DIV_T_PAY`, `SUM_FWD_DIVS`, `SUM_REALIZED_DIVS` are documented in §4.2 (they are used inside / alongside a `schedule_div` block).

3.12 Closed-form analytics (inner valuations)

- `BS_EURO_CALL(F, K, vol, t, df)` / `BS_EURO_PUT(F, K, vol, t, df)` — Black–Scholes European price from forward `F`, strike `K`, vol, time `t`, and discount factor `df`. For an inner option inside a structured payoff. `receive BS_EURO_CALL(FORWARD(SPX, T), 100, 0.2, T, DF("USD", T))`

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2), \quad d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

- `BACHELIER_EURO_CALL(F, K, vol, t, df)` / `BACHELIER_EURO_PUT(...)` — normal-model European price (rates options). `vol` is a normal (absolute) vol.

$$C = e^{-rT}\left[(F - K)N(d) + \sigma\sqrt{T}\,\varphi(d)\right], \quad d = \frac{F - K}{\sigma\sqrt{T}}, \quad F = Se^{rT}$$

3.13 Selection, control & lookup

- `where(cond, a, b)` — element-wise select; identical to the ternary `a if cond else b`. Alias `WHERE`. `receive where(perf >= cb, cpn, 0.0)`
- `if_(cond, a, b)` — select alias (`IF_`, `ifelse`, `IFELSE`). `receive if_(ki > 0.5, perf, 1.0)`
- `piecewise(conds, values)` — first-true multi-branch select over equal-length lists. `receive piecewise([c1, c2], [v1, v2])`
- `SNAPSHOT(label, value)` — store `value` into per-path state under `label` **at this event** and return it; the way to capture a simulated level (e.g. spot at t_0) for reuse later. `at 0.0 { set s0 = SNAPSHOT("s0", SPX) }`
- `PAST_FIX(name, when=None)` — historical observed fixing from the bound `MarketData` at date/tenor/year-fraction `when` (negative = past; `None` = the current event date). Alias `FIXING`. Requires `market_data` at compile time. `receive SPX / PAST_FIX("SPX", "2025-01-01")`
- `FIX(name, …)` — dispatcher: returns the current snapshot value for an equity/FX/rate-factor name, or routes rate helpers (`FIX("LIBOR", …)`, `FIX("DF", …)`, `FIX("FORWARD", …)`). `set s = FIX("SPX")`

3.14 Additional credit, basket & statistics primitives

These functions are part of the engine vocabulary and round out the credit, multi-currency basket, and running-statistics families above.

Credit (single-name and pool).

- `DEFAULTED(issuer)` — `1 − IS_ALIVE(issuer)`; 1 once the issuer has defaulted. `set d = DEFAULTED("ACME")`
- `INTENSITY(issuer)` — alias of `HAZARD(issuer)`; the instantaneous default intensity `λ(t)`. `set lam = INTENSITY("ACME")`
- `FWD_SURVIVAL(issuer, T)` — the path-conditional forward survival probability $\hat{S}(t_{now}, T)$ to horizon `T` given the current credit state. Alias `Q_SURVIVE`. `set q = FWD_SURVIVAL("ACME", 5.0)`
- `FWD_SURVIVAL_BOND(issuer, T)` — the bond-measure forward survival (recovery-adjusted variant used in defaultable-bond PVs). Alias `Q_SURVIVE_BOND`. `set qb = FWD_SURVIVAL_BOND("ACME", 5.0)`
- `BASKET_LOSS(names, weights=None)` — weighted portfolio loss $\Sigma\, w_i \cdot (1 - R_i) \cdot (1 - A_i)$ over the obligors `names` (loss-given-default times default indicator). `set L = BASKET_LOSS(book, [0.5, 0.5])`
- `FIRST_TO_DEFAULT(names)` — 1 if **any** obligor in `names` has defaulted (first-to-default trigger). `set ftd = FIRST_TO_DEFAULT(book)`
- `NUM_DEFAULTED(names)` — count of defaulted obligors in `names` (per-path float); use for nth-to-default products via `NUM_DEFAULTED(pool) >= n`. `set k = NUM_DEFAULTED(book)`

(`IS_ALIVE`, `RECOVERY`, `HAZARD`, `JUST_DEFAULTED`, `BOND`, `TRANCHE_LOSS`, `INDEX_SPREAD_NOTIONAL` are documented in §3.10.)

Multi-currency / performance baskets.

- `ABSOLUTE_BASKET(weights, components, currency=None)` — price basket $\Sigma\, w_i \cdot S_i(t) \cdot FX_i(t)$, converting each component from its own currency into the basket `currency` (default = deal currency); same-currency components are a no-op. FX is resolved by triangulation through the simulated FX factors. `set b = ABSOLUTE_BASKET([0.5, 0.5], [SPX, SX5E], currency="USD")`
- `RELATIVE_BASKET(weights, components, currency=None, convert_fx=False)` — performance basket $\Sigma\, w_i \cdot S_i(t)/S_{i,0}$ (dimensionless, default). With `convert_fx=True` it includes the FX return $\Sigma\, w_i \cdot (S_i(t) \cdot FX_i(t))/(S_{i,0} \cdot FX_i(0))$, i.e. an unhedged / quanto-compo performance basket. `set p = RELATIVE_BASKET([0.5, 0.5], [SPX, SX5E])`

Accrued performance & risk statistics.

- `PERF_ACCRUED(asset, type="relative", K=1.0, cap=None, floor=None)` — like `PERF`, but **gated to a schedule window**: returns 0 if the current date is before the first fixing or after the last fixing of the accrual schedule. `set r = PERF_ACCRUED(SPX, cap=0.1, floor=-0.1)`
- `RATIO_ACCRUED(asset, cap=None, floor=None)` — the ratio form of `PERF_ACCRUED` (schedule-window-gated `RATIO`). `set r = RATIO_ACCRUED(SPX)`
- `SHARPE_RATIO(label, value, annualization=252, rf=0.0)` — the running, annualised Sharpe ratio of `value`'s log-returns, accumulated under the string `label`; `rf` is the per-annum risk-free rate subtracted from the mean. Aliases `SHARPE`, `SHARP_RATIO`. `set sr = SHARPE_RATIO("sr", SPX, 252, 0.02)`

> **British / American spellings.** Every `REALIZED_*` function has an identical `REALISED_*` alias (`REALISED_VAR`, `REALISED_VOL`, `REALISED_QV`, `REALISED_QV_CROSS`, and their `_IF` variants), and the long forms `REALIZED_VARIANCE` / `REALISED_VARIANCE`, `REALIZED_VOLATILITY` / `REALISED_VOLATILITY`, `REALIZED_QUADRATIC_VARIATION` / `REALISED_QUADRATIC_VARIATION` are accepted. Combined with case-insensitivity, `realised_vol`, `REALISED_VOL`, and `Realized_Volatility` all resolve to the same function.

3.15 Typed signature index (argument names & types)

*Argument **names** are the canonical DSL parameters (from the function descriptions above and the curated market/credit/accumulator forms); argument **types** are: `number`, `int`, `date` (year-fraction or tenor), `tenor` (a tenor token like `6M`), `string`, `list`, `bool`, `condition`. `…` = variadic; `name: type = default` is an optional **keyword** argument (only functions in the engine's kwarg whitelist); `[name: type]` is an optional **positional** argument. Every name below is parse-checked against the engine. Auto-generated by `tools/gen_typed_signatures.py`.*

Conditional

function	typed signature
`IF`	`IF(cond: condition, then: number, else_: number)`
`IFELSE`	`IFELSE(cond: condition, then: number, else_: number)`
`IF_`	`IF_(cond: condition, then: number, else_: number)`
`WHERE`	`WHERE(…)` *(variadic / positional, arity (1, 3))*

Credit index / tranche

function	typed signature
`ABSOLUTE_BASKET`	`ABSOLUTE_BASKET(weights: list, components: number, [currency: string])`
`BASKET_LOSS`	`BASKET_LOSS(names: list, [weights: list])`
`BOND`	`BOND(coupon_rate: number = …, maturity: date = …, frequency: tenor = …, notional: number = 1.0, issuer_name: number = …, recovery_rate: number = …)`
`CDO_VALUE`	`CDO_VALUE(index: string, attach: number, detach: number, spread: number, schedule: list, config: string, effective: date, upfront: number = None, is_upfront_payer: bool = True, nominal_exchange: bool = False)`
`FWD_SURVIVAL_BOND`	`FWD_SURVIVAL_BOND(issuer: string, T: number)`
`INDEX_SPREAD_NOTIONAL`	`INDEX_SPREAD_NOTIONAL(name, ...: number, weights: list)`
`RELATIVE_BASKET`	`RELATIVE_BASKET(weights: list, components: number, [currency: string], [convert_fx: number])`
`TRANCHE_LOSS`	`TRANCHE_LOSS(name, ...: number, attach: number, detach: number, weights: list, notional: number)`

Credit per-name / survival

function	typed signature
BARRIER_SURVIVAL	BARRIER_SURVIVAL('label': number, value: number, level[, monitoring: number = , sigma=, ...)
CORRIDOR_SURVIVAL	CORRIDOR_SURVIVAL('label': number, value: number, lo: number, hi[, monitoring: number = , ...)
DEFAULTED	DEFAULTED(name: string)
FIRST_TO_DEFAULT	FIRST_TO_DEFAULT(name, ...: number)
FWD_SURVIVAL	FWD_SURVIVAL(name: string, t_start: date, t_end: date)
HAZARD	HAZARD(name: string)
INTENSITY	INTENSITY(name: string)
IS_ALIVE	IS_ALIVE(name: string)
JUST_DEFAULTED	JUST_DEFAULTED(name: string, prev_alive: list)
NUM_DEFAULTED	NUM_DEFAULTED(name, ...: number)
Q_SURVIVE	Q_SURVIVE(name: string, t_start: date, t_end: date)
RECOVERY	RECOVERY(name: string)

Dividends

function	typed signature
DIV_AMOUNT	DIV_AMOUNT(asset: string)
FWD_DIV_AMOUNT	FWD_DIV_AMOUNT(asset: string)

Inflation

function	typed signature
YOY_INFLATION	YOY_INFLATION(index: string, t_start: date, t_end: date)
ZC_INFLATION_SWAP_RATE	ZC_INFLATION_SWAP_RATE(index: string, t_start: date, t_end: date)

Math / scalar

function	typed signature
ABS	ABS(x: number)
BACHELIER_EURO_CALL	BACHELIER_EURO_CALL(F: number, K: number, vol: number, t: date, df: number)
BACHELIER_EURO_PUT	BACHELIER_EURO_PUT(…)
BS_EURO_CALL	BS_EURO_CALL(F: number, K: number, vol: number, t: date, df: number)
BS_EURO_PUT	BS_EURO_PUT(F: number, K: number, vol: number, t: date, df: number)
CEIL	CEIL(x: number)
CLIP	CLIP(x: number, lo: number, hi: number)
EXP	EXP(x: number)

function	typed signature
EXPM1	EXPM1(x: number)
FLOOR	FLOOR(x: number)
INTERP	INTERP(x: number, xs: number, ys: number)
LERP	LERP(x: number, x0: number, x1: number, y0: number, y1: number)
LINTERP	LINTERP(x: number, x0: number, x1: number, y0: number, y1: number)
LN	LN(x: number)
LOG	LOG(x: number)
LOG10	LOG10(x: number)
LOG1P	LOG1P(…) *(variadic / positional, arity (1, 2))*
LOG2	LOG2(…) *(variadic / positional, arity (1, 2))*
POW	POW(x: number, n: int)
SIGN	SIGN(x: number)
SQRT	SQRT(x: number)

Order statistics (rainbow)

function	typed signature
ARGMAX_OF	ARGMAX_OF(group: list)
ARGMIN_OF	ARGMIN_OF(group: list)
BEST	BEST(group: list)
BEST_OF	BEST_OF(group: list)
BEST_OF_EXCLUDING	BEST_OF_EXCLUDING(group: list, exclude: number, min_keep: number = 0)
BEST_OF_IDX	BEST_OF_IDX(group: list)
BEST_OF_IDX_EXCLUDING	BEST_OF_IDX_EXCLUDING(group: list, exclude: number, min_keep: number = 0)
EXCLUDE_CONT_BEST	EXCLUDE_CONT_BEST(group: list, exclude: number, min_keep: number = 0)
EXCLUDE_CONT_WORST	EXCLUDE_CONT_WORST(group: list, exclude: number, min_keep: number = 0)
KTH_BEST	KTH_BEST(group: list, k: int)
KTH_WORST	KTH_WORST(group: list, k: int)
NTH_BEST	NTH_BEST(group: list, n: int)
NTH_WORST	NTH_WORST(group: list, n: int)
RANK	RANK(…) *(variadic / positional, arity (1, None))*
RANK_OF	RANK_OF(group: list, member: number)
SUM_BEST	SUM_BEST(…) *(variadic / positional, arity (1, None))*
SUM_BEST_K	SUM_BEST_K(group: list, k: int)
SUM_WORST	SUM_WORST(…) *(variadic / positional, arity (1, None))*
SUM_WORST_K	SUM_WORST_K(group: list, k: int)
WORST	WORST(group: list)
WORST_OF	WORST_OF(group: list)
WORST_OF_EXCLUDING	WORST_OF_EXCLUDING(group: list, exclude: number, min_keep: number = 0)

function	typed signature
WORST_OF_IDX	WORST_OF_IDX(group: list)
WORST_OF_IDX_EXCLUDING	WORST_OF_IDX_EXCLUDING(group: list, exclude: number, min_keep: number = 0)

Path accumulators (state-label first)

function	typed signature
INF	INF(label: string, value: number [, cond: condition], init: number = 0.0)
PERF	PERF(asset: string, type: number = "relative", K: number = 1.0, cap: number = None, floor: number = None)
PERF_ACCRUED	PERF_ACCRUED(asset: string, type: number = "relative", K: number = 1.0, cap: number = None, floor: number = None)
PERF_PROD	PERF_PROD(asset: string, …)
PERF_PROD_IF	PERF_PROD_IF(label: string, value: number, cond: condition, init: number = 0.0)
PERF_SUM	PERF_SUM(asset: string, K: number = 1.0, cap: number = None, floor: number = None)
PERF_SUM_IF	PERF_SUM_IF(asset: string, cond: condition, …)
RATIO	RATIO(all: list, reset: bool = false)
RATIO_ACCRUED	RATIO_ACCRUED(asset: string, cap: number = None, floor: number = None)
RATIO_PROD	RATIO_PROD(asset: string, …)
RATIO_PROD_IF	RATIO_PROD_IF(label: string, value: number, cond: condition, init: number = 0.0)
RATIO_SUM	RATIO_SUM(asset: string, reset: bool = false, cap: number = None, floor: number = None)
RATIO_SUM_IF	RATIO_SUM_IF(asset: string, cond: condition, …)
REALISED_QUADRATIC_VARIATION	REALISED_QUADRATIC_VARIATION(label: string, value: number [, cond: condition], init: number = 0.0)
REALISED_QUADRATIC_VARIATION_IF	REALISED_QUADRATIC_VARIATION_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALISED_QV	REALISED_QV(label: string, value: number [, cond: condition], init: number = 0.0)
REALISED_QV_CROSS	REALISED_QV_CROSS(label: string, value: number [, cond: condition], init: number = 0.0)
REALISED_QV_IF	REALISED_QV_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALISED_VAR	REALISED_VAR(label: string, value: number [, cond: condition], init: number = 0.0)
REALISED_VARIANCE	REALISED_VARIANCE(label: string, value: number [, cond: condition], init: number = 0.0)

function	typed signature
REALISED_VARIANCE_IF	REALISED_VARIANCE_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALISED_VAR_IF	REALISED_VAR_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALISED_VOL	REALISED_VOL(label: string, value: number [, cond: condition], init: number = 0.0)
REALISED_VOLATILITY	REALISED_VOLATILITY(label: string, value: number [, cond: condition], init: number = 0.0)
REALISED_VOLATILITY_IF	REALISED_VOLATILITY_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALISED_VOL_IF	REALISED_VOL_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALIZED_QUADRATIC_VARIATION	REALIZED_QUADRATIC_VARIATION(label: string, value: number [, cond: condition], init: number = 0.0)
REALIZED_QUADRATIC_VARIATION_IF	REALIZED_QUADRATIC_VARIATION_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALIZED_QV	REALIZED_QV(label: string, value: number, annualization: number = 252)
REALIZED_QV_CROSS	REALIZED_QV_CROSS(label: string, value_x: number, value_y: number, annualization: number = 252)
REALIZED_QV_IF	REALIZED_QV_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALIZED_VAR	REALIZED_VAR(label: string, value: number, annualization: number = 252)
REALIZED_VARIANCE	REALIZED_VARIANCE(label: string, value: number [, cond: condition], init: number = 0.0)
REALIZED_VARIANCE_IF	REALIZED_VARIANCE_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALIZED_VAR_IF	REALIZED_VAR_IF(label: string, value: number, cond: condition, annualization: number = 252)
REALIZED_VOL	REALIZED_VOL(label: string, value: number, annualization: number = 252)
REALIZED_VOLATILITY	REALIZED_VOLATILITY(label: string, value: number [, cond: condition], init: number = 0.0)
REALIZED_VOLATILITY_IF	REALIZED_VOLATILITY_IF(label: string, value: number, cond: condition, init: number = 0.0)
REALIZED_VOL_IF	REALIZED_VOL_IF(label: string, value: number, cond: condition, init: number = 0.0)

function	typed signature
RUNNING_AVG	RUNNING_AVG(label: string, value: number [, cond: condition], init: number = 0.0)
RUNNING_AVG_IF	RUNNING_AVG_IF(label: string, value: number, cond: condition, init: number = 0.0)
RUNNING_COUNT	RUNNING_COUNT(label: string, init: number = 0.0)
RUNNING_COUNT_IF	RUNNING_COUNT_IF(label: string, cond: condition, init: number = 0.0)
RUNNING_MAX	RUNNING_MAX(label: string, value: number, init: number = None)
RUNNING_MEAN	RUNNING_MEAN(label: string, value: number, init: number = 0.0)
RUNNING_MEAN_IF	RUNNING_MEAN_IF(label: string, value: number, cond: condition, init: number = 0.0)
RUNNING_MIN	RUNNING_MIN(label: string, value: number, init: number = None)
RUNNING_PROD	RUNNING_PROD(label: string, value: number, init: number = 1.0)
RUNNING_PRODUCT	RUNNING_PRODUCT(label: string, value: number [, cond: condition], init: number = 0.0)
RUNNING_PRODUCT_IF	RUNNING_PRODUCT_IF(label: string, value: number, cond: condition, init: number = 0.0)
RUNNING_PROD_IF	RUNNING_PROD_IF(label: string, value: number, cond: condition, init: number = 0.0)
RUNNING_SUM	RUNNING_SUM(label: string, value: number, init: number = 0.0)
RUNNING_SUM_IF	RUNNING_SUM_IF(label: string, cond: condition, value: number, init: number = 0.0)
RUNNING_SUM_LOG	RUNNING_SUM_LOG(label: string, value: number, init: number = 0.0)
RUNNING_SUM_LOG_IF	RUNNING_SUM_LOG_IF(label: string, value: number, cond: condition, init: number = 0.0)
SHARPE	SHARPE(label: string, value: number [, cond: condition], init: number = 0.0)
SHARPE_RATIO	SHARPE_RATIO(label: string, value: number, annualization: number = 252, rf: number = 0.0)
SHARP_RATIO	SHARP_RATIO(label: string, value: number [, cond: condition], init: number = 0.0)
SNAPSHOT	SNAPSHOT(label: string, asset: string)
SUP	SUP(label: string, value: number [, cond: condition], init: number = 0.0)

Payoff primitives

function	typed signature
CALL	CALL(value: number, strike: number)
CALL_PAYOFF	CALL_PAYOFF(value: number, strike: number)
CALL_SPREAD	CALL_SPREAD(value: number, lo: number, hi: number)
CAPPED_AT	CAPPED_AT(x: number, cap: number)
CAP_AT	CAP_AT(x: number, cap: number)
CLAMP	CLAMP(x: number, lo: number, hi: number)
COLLAR	COLLAR(x: number, floor: number, cap: number)
DIGITAL	DIGITAL(value: number, strike: number)
DOUBLE_DIGITAL	DOUBLE_DIGITAL(value: number, lo: number, hi: number)
FLOORED_AT	FLOORED_AT(x: number, floor: number)
FLOOR_AT	FLOOR_AT(x: number, floor: number)
HEAVISIDE	HEAVISIDE(x: number)
INDIC	INDIC(cond: condition)
INDICATOR	INDICATOR(cond: condition)
INDICATOR_OF	INDICATOR_OF(cond: condition)
LADDER	LADDER(value: number, levels: list)
NEG	NEG(…) *(variadic / positional, arity (1, 1))*
NEGATIVE_PART	NEGATIVE_PART(…) *(variadic / positional, arity (1, 1))*
PIECEWISE	PIECEWISE(…) *(variadic / positional, arity (3, 3))*
POS	POS(…) *(variadic / positional, arity (1, 1))*
POSITIVE_PART	POSITIVE_PART(…) *(variadic / positional, arity (1, 1))*
PUT	PUT(value: number, strike: number)
PUT_PAYOFF	PUT_PAYOFF(value: number, strike: number)
RAMP	RAMP(…) *(variadic / positional, arity (1, 1))*
RANGE_DIGITAL	RANGE_DIGITAL(…) *(variadic / positional, arity (3, 3))*
SIGMOID	SIGMOID(…) *(variadic / positional, arity (1, 2))*
SMOOTH_INDICATOR	SMOOTH_INDICATOR(…) *(variadic / positional, arity (2, 5))*
SMOOTH_STEP	SMOOTH_STEP(…) *(variadic / positional, arity (2, 5))*
STEP	STEP(…) *(variadic / positional, arity (1, 1))*
STRADDLE	STRADDLE(…) *(variadic / positional, arity (2, 2))*

Rates & fixings

function	typed signature
CMS	CMS(currency: string, t_start: date, n_payments: int, freq: tenor = 1.0, tenor: tenor = None)
DF	DF(currency: string, t_pay: date)
FIX	FIX(name: string, value[, ...: number)
FIXING	FIXING(name: string, when: number)
FORWARD	FORWARD(asset: string, t: date)

function	typed signature
LIBOR	LIBOR(currency: string, t_start: date, t_end: date, tenor: tenor = None)
OIS	OIS(index: string, t_start: date, t_end: date)
PAST_FIX	PAST_FIX(name: string, when: number)
RFR	RFR(currency: string, t_start: date, t_end: date, index: string = None)
SWAP	SWAP(currency[, ...: number)

Reductions & statistics

function	typed signature
ALL	ALL(…: ...)
ANY	ANY(…: ...)
AVERAGE	AVERAGE(group: list)
AVG	AVG(group: list)
AVG_EXCLUDING	AVG_EXCLUDING(group: list, exclude: number, min_keep: number = 0)
BASKET	BASKET(weights: list, assets: list)
COUNT	COUNT(c1, c2, …: number)
MAX	MAX(a: number, b: number, …)
MAXIMUM	MAXIMUM(…) *(variadic / positional, arity (0, None))*
MEAN	MEAN(…) *(variadic / positional, arity (0, None))*
MEDIAN	MEDIAN(group: list)
MEDIAN_EXCLUDING	MEDIAN_EXCLUDING(…)
MIN	MIN(a: number, b: number, …)
MINIMUM	MINIMUM(…) *(variadic / positional, arity (0, None))*
PCTRANK	PCTRANK(…) *(variadic / positional, arity (1, None))*
PCT_RANK	PCT_RANK(group: list, x: number)
PERCENTILE	PERCENTILE(group: list, q: number)
PERCENTILE_RANK	PERCENTILE_RANK(…) *(variadic / positional, arity (1, None))*
PROD	PROD(a: number, b: number, …)
PRODUCT	PRODUCT(a: number, b: number, …)
QUANTILE	QUANTILE(group: list, q: number)
RANGE	RANGE(group: list)
RMS	RMS(group: list)
SOFTMAX	SOFTMAX(…) *(variadic / positional, arity (1, None))*
SOFTMIN	SOFTMIN(…) *(variadic / positional, arity (1, None))*
SOFT_MAX	SOFT_MAX(…) *(variadic / positional, arity (1, None))*
SOFT_MIN	SOFT_MIN(…) *(variadic / positional, arity (1, None))*
STD	STD(…) *(variadic / positional, arity (0, None))*
STDDEV	STDDEV(…) *(variadic / positional, arity (0, None))*
STDDEV_EXCLUDING	STDDEV_EXCLUDING(…) *(variadic / positional, arity (2, 3))*
STDEV	STDEV(group: list)
STDEV_EXCLUDING	STDEV_EXCLUDING(…)

function	typed signature
`STD_EXCLUDING`	`STD_EXCLUDING(…)` *(variadic / positional, arity (2, 3))*
`SUM`	`SUM(a: number, b: number, …)`
`SUM_EXCLUDING`	`SUM_EXCLUDING(…)`
`SUM_LOG`	`SUM_LOG(a: number, b: number, …)`
`VAR`	`VAR(group: list)`
`VARIANCE`	`VARIANCE(group: list)`
`VARIANCE_EXCLUDING`	`VARIANCE_EXCLUDING(…)` *(variadic / positional, arity (2, 3))*
`VAR_EXCLUDING`	`VAR_EXCLUDING(…)`

Part 4 — Dates, schedules, and iterators

Schedule key reference — `schedule(...)` / `schedule_div(...)` (typed)

Valid keys: `start, end, n, frequency, freq, stub, dates, calendar, bda, day_count, dcc` (aliases: `freq` == `frequency`, `day_count` == `dcc`).

key	type	notes
`start`	date-atom	tenor `3M` or ISO date; schedule anchor
`end`	date-atom	tenor `5Y` or ISO date; schedule end
`freq / frequency`	tenor token	`6M`, `3M`, `1Y` …
`n`	int	number of periods (alt. to `end`)
`day_count / dcc`	string	`"ACT/365"`, `"30/360"` …
`stub`	token	stub handling
`dates`	[date,…]	explicit date list (alt. to start/end/freq)
`calendar`	token	holiday calendar
`bda`	token	business-day adjustment
`underlying`	string	(`schedule_div`) dividend underlying
`drive_by`	string	(`schedule_div`) `ex` or `pay` date driver
`obs_time`	string	(`schedule_div`) observation-time convention
`include_first / include_last`	bool	(`schedule_div`) include endpoints

Example (parse-verified): `for t in schedule(start=3M, end=5Y, freq=6M, day_count="ACT/365") { at t { receive 1.0 } }`. Endpoints are **inclusive**.

A date in an `at <date>` slot (or a `schedule` bound) may be written in any of these forms, all resolved against `as_of`:

Form	Example	Meaning
float (year fraction)	`1.5`	1.5 years from `as_of`

Form	Example	Meaning
tenor	`6M`, `3M`, `2W`, `5D`, `1Y`	calendar tenor (`Y A M W D Q S` units)
compound / chained tenor	`1Y+6M`, `1Y6M`, `6M+1D`	sum of tenor chunks
ISO date	`"2027-06-12"`	absolute calendar date
quoted form of any of the above	`"18M"`, `"1.5"`	same atom, quoted (matches `as_of`)
`t0` (name)	`t0`	year-fraction 0, i.e. the `as_of` date
bare loop variable	`t` (inside `for t in …`)	the current iteration's date
negative / past tenor	`-3M`, `-0.5`	a date *before* `as_of` (past fixings)

> **No date arithmetic in the slot.** The `at` slot takes a bare **name** (`t`, `t0`) or an **absolute** date/tenor — it does **not** accept arithmetic on a name. `at t-3M`, `at t + 0.5`, and `at (t - 0.25)` all fail to parse. To offset relative to the loop date, do the arithmetic **inside the payoff expression** instead: write `at t { receive f(t - 0.25) }` — expressions like `t - tau` are fine on the right of `receive`/`pay`/`set`, just not in the date slot itself.

4.1 `schedule(...)` — periodic date generator

`schedule` has **two distinct forms**, and they are easy to confuse:

(a) Iterator form — `for <var> in schedule(...) { ... }`. Generates a date sequence the `for` loop walks. Keys (all optional except the bounds):

Key (aliases)	Meaning	Example
`start`	first date (float / tenor / ISO)	`start=0.0`
`end`	last date	`end=5.0`
`freq` / `frequency`	period between dates	`freq="6M"`
`n`	number of periods (alternative to `freq`)	`n=10`
`stub`	stub handling (`"short_front"`, `"long_back"`, …)	`stub="short_front"`
`dates`	an explicit date list (overrides start/end/freq)	`dates=[0.5, 1.0, 1.5]`
`calendar`	holiday calendar name	`calendar="TARGET"`
`bda`	business-day adjustment (`"following"`, `"modfollowing"`, …)	`bda="modfollowing"`
`day_count` / `dcc`	day-count convention for accrual	`day_count="ACT/360"`

```
for t in schedule(start=0.0, end=5.0, freq="6M", day_count="ACT/360") {
  at t { receive tau * LIBOR("USD", t, t + 0.5) }
}
```

(b) Block form — `schedule(...) { ... }` (no `for`). The same keys, but the block body is emitted once per generated date with the loop variable available as the event date; used when you want the schedule to drive the events directly rather than binding a named variable.

4.2 `schedule_div(...)` — discrete-dividend schedule

`schedule_div` iterates over the **discrete-dividend schedule of a named underlying**, generating one event per dividend in a `[start, end]` window. It requires `market_config` (carrying the underlying's `DividendSpec`) to be passed at compile time. Write it as a `for` iterator — `for <var> in schedule_div(...) { … }` — symmetric with `for … in schedule(...)`; the loop variable binds to each dividend's driving date (see "Iterator form" below). A standalone-block form without a loop variable, `schedule_div(...) { … }`, is also accepted as a shorthand when the body never needs the date. Keys:

Key	Meaning	Default
`underlying`	the equity whose dividends to iterate	— (required)
`start` / `end`	ex-date window (always filtered by ex-date)	—
`include_first` / `include_last`	include the boundary dividends	`True` / `True`
`drive_by`	when the flow fires: `"ex_date"` or `"pay_date"` (controls discounting)	`"pay_date"`
`obs_time`	observation time used by `FWD_DIV_AMOUNT`	`t0`

Inside a `schedule_div` body, these **dividend macros** are available:

Macro	Signature	Meaning
`DIV_AMOUNT`	`DIV_AMOUNT(asset)`	path-realised dividend $C_i + Y_i \cdot S_{t_{ex}^-}$ (cum-div spot captured automatically)
`FWD_DIV_AMOUNT`	`FWD_DIV_AMOUNT(asset)`	compile-time deterministic $C_i + Y_i \cdot F^S(obs\_time, t_{ex}^-)$ from the analytic forward
`DIV_C`	`DIV_C` or `DIV_C(asset)`	the current dividend's cash component C_i (bare scalar; the call form is accepted and lowers to the bare token)
`DIV_Y`	`DIV_Y` or `DIV_Y(asset)`	the current dividend's proportional (yield) component Y_i (bare scalar; call form accepted)
`DIV_T_EX`	`DIV_T_EX` or `DIV_T_EX(asset)`	ex-date of the current dividend
`DIV_T_PAY`	`DIV_T_PAY` or `DIV_T_PAY(asset)`	pay-date of the current dividend
`current`	`current` (bare)	the **0-based index of the current dividend** in the window (the per-dividend counter)
`SIZE`	`SIZE` (bare)	the **total number of dividends** the schedule expands to

Because `DIV_C` / `DIV_Y` are bare scalars they **compose in arithmetic**: `DIV_C(SPX) + DIV_Y(SPX) * SPX` is a valid expression (the call form is a convenience that lowers to the bare identifiers `DIV_C` / `DIV_Y`). `DIV_AMOUNT` and `FWD_DIV_AMOUNT` genuinely need the asset argument and stay call-form.

`current` and `SIZE` — the per-dividend counter. A `schedule_div`'s dividend dates come from the market's dividend spec, so they are **not** known at lowering (a compile-time `mutable` counter is therefore impossible here, and is rejected). Instead the engine substitutes two index tokens **per dividend at expansion time**, when the dates are known: `current` (the 0-based dividend index, i.e. the counter you would otherwise want a mutable for) and `SIZE` (the dividend count). Both are bare identifiers, so they **compose in arithmetic and as vector indices** — e.g. `receive coupons[current]` with `let coupons[K] = [...]` gives a per-dividend table lookup, and `current / (SIZE - 1)` gives a 0→1 progress ramp. (`SIZE(x)` as a **call** still folds at compile time for the enumerable iterables in the static-vector section; only the *bare* `SIZE` token inside a `schedule_div` body is the engine-substituted dividend count.)

And two **aggregate** dividend macros that work **outside** any `schedule_div` block:

Macro	Signature	Meaning
`SUM_FWD_DIVS`	`SUM_FWD_DIVS(asset, start, end)`	compile-time scalar Σ_i `(C_i + Y_i·F^S(0, t_ex,i⁻))` for ex-dates in the window
`SUM_REALIZED_DIVS`	`SUM_REALIZED_DIVS(asset, start, end)`	path-dependent realised-dividend sum (auto-injects a hidden `schedule_div + RUNNING_SUM`); matches `SUM_FWD_DIVS` in expectation

Example — a dividend swap leg (pays the realised dividends collected over 3y):

```
product DivSwap {
  currency = "USD"
  underlyings = [SPX]
  state { acc = 0.0 }
  events {
    for t in schedule_div(underlying=SPX, start=0.0, end=3.0, drive_by=pay_date) {
      set acc = RUNNING_SUM("acc", DIV_AMOUNT(SPX))
    }
    at 3.0 { only_last receive acc }
  }
}
```

Example — using `DIV_C` / `DIV_Y` explicitly (split the cash and proportional parts, e.g. to pay only the yield component):

```
for t in schedule_div(underlying=SPX, start=0.0, end=2.0) {
  receive DIV_Y * SPX        # proportional part only (Y_i · S_ex)
  set cash = RUNNING_SUM("c", DIV_C)   # accumulate the fixed cash part
}
```

Iterator form — the loop-variable binding. In `for <var> in schedule_div(...)` the loop variable binds to the current dividend's **driving date**, and the body event fires at that instant, so referencing the underlying samples the spot there:

- `drive_by = ex_date` → the loop variable binds to `t_ex − 1e-9`, the **cum-div instant** (just before the dividend drop), so the underlying inside the body is the **pre-drop** spot. This makes the component reconstruction **exact**: `DIV_C(asset) + DIV_Y(asset) · asset == DIV_AMOUNT(asset)`.
- `drive_by = pay_date` → the loop variable binds to `t_pay` (after the drop, as paid).

```
product DivSwap {
  currency = "USD"
  underlyings = [SPX]
  state { realised = 0.0 }
  events {
    # t binds to t_ex − 1e-9; SPX is the pre-drop (cum-div) spot, so
    # DIV_C + DIV_Y * SPX reconstructs DIV_AMOUNT exactly.
    for t in schedule_div(underlying=SPX, start=0.0, end=3.0, drive_by=ex_date) {
      set realised = realised + (DIV_C(SPX) + DIV_Y(SPX) * SPX)
    }
    at 3.0 { receive realised }
  }
}
```

The standalone-block form (no loop variable) behaves identically except that no date variable is bound; the loop-variable binding and event-instant shift apply only to the `for … in schedule_div(...)` form.

Example — forward dividends as a single scalar (dividend-futures style):

```
at 3.0 { receive SUM_FWD_DIVS(SPX, 0.0, 3.0) }
```

4.3 `continuous(...)` — fine monitoring grid

`continuous(start, end, steps=N)` produces a uniform fine grid of `N` points — the monitoring grid for continuous-barrier / Brownian-bridge products:

```
for u in continuous(0.0, 3.0, steps=200) {
  at u { set surv = BARRIER_SURVIVAL("ki", SPX, 60.0, "down",
                                     monitoring="continuous", sigma=0.25) }
}
```

4.4 Generic event types (engine-level, top-level events)

A handful of event types are **engine-level**: they are written as a top-level event `<type> { date = …; … }` in the `events` block (not as a statement inside an `at { }`). The most useful is the undiscounted payment.

Event type	Form	Meaning
`pay_undisc`	`pay_undisc { date = …; amount = … }` (top-level event), **or** the `receive_undisc` / `pay_undisc` statements inside an `at { }` block (see §4.5)	pay `amount` **without discounting** (`discounted = False`)
`american`	`american { … }`	American/Bermudan optimal-stopping (sugar over `decide`)
`convertible`	`convertible { … }`	convertible-bond holder decision
`bond_physical_exercise`, `swap_physical_exercise`, `fx_physical_exercise`, `equity_physical_exercise`	`<type> { … }`	physical-settlement exercise into a declared instrument

Example — an undiscounted payment (pays the intrinsic at 1y with no discount factor applied):

```
product Undisc {
  currency = "USD"
  underlyings = [SPX]
  events {
    pay_undisc { date = 1.0; amount = max(SPX - 100.0, 0) }
  }
}
```

4.5 `receive_undisc` / `pay_undisc` statements (undiscounted cashflows)

In addition to the top-level `pay_undisc { … }` event (§4.4), the DSL provides two **statement** keywords usable directly inside an `at { }` block, exactly like `receive` / `pay` but **without discounting**:

Statement	Sign	Discounting	Example
`receive <expr>`	+ to holder	discounted	`receive max(SPX - 100.0, 0)`
`pay <expr>`	− (holder pays)	discounted	`pay premium`
`receive_undisc <expr>`	+ to holder	**undiscounted**	`receive_undisc 0.5`
`pay_undisc <expr>`	− (holder pays)	**undiscounted**	`pay_undisc 0.2`

Each undiscounted statement emits a standalone `pay_undisc` engine event (`discounted = False`). Unlike consecutive `receive`/`pay` statements — which net into a single discounted cashflow at the instant — an undiscounted statement is always its own event, so discounted and undiscounted legs can coexist in the same `at { }`:

```
product MixedDiscounting {
  currency = "USD"
  discount = 0.02
  underlyings = [SPX]
  events {
    at 3.0 {
      receive max(SPX - 100.0, 0)   # discounted intrinsic
      receive_undisc 0.5             # paid in full, no discount factor
      pay_undisc 0.2                 # holder pays 0.2 undiscounted
    }
  }
}
```

When to use undiscounted payments: for quantities that are *already* discounted (so a second discount factor would double-count), for forward-valued legs, or for payoffs expressed in a rebased numéraire. For example, a par redemption written `receive_undisc 1.0` at maturity pays exactly `1.0`, whereas `receive 1.0` pays `1.0 × DF(0, T)`.

Cross-currency. All four cashflow statements (`receive`, `pay`, `receive_undisc`, `pay_undisc`) accept an optional `currency` (and `fx`) for a foreign-denominated cashflow — either as a block field (`receive { amount = …; currency = "EUR" }`) or as a trailing suffix (`receive <expr> currency "EUR"`). See the cross-currency (quanto / compo) section under the statements list for the FX-conversion semantics.

Part 5 — Fully-worked payoffs

Each of the following has been compiled and priced; they exercise the operators, the `for` loop, the flow flags, and a cross-section of the function vocabulary.

5.1 European call — operators only

```
product European {
  currency = "USD"
  discount = 0.02
  underlyings = [SPX]
  events {
    at 1.0 { receive max(SPX - 100.0, 0) }
  }
}
```

5.2 Worst-of autocallable — `for`, flags, `exercise`, ternary, `INF`, `RATIO`

```
product Autocallable {
  currency = "USD"
  discount = 0.02
  underlyings = [SPX, NDX]
  groups all = [SPX, NDX]
  let cpn = 0.06
  let ab = 1.0
  let cb = 0.7
  let kib = 0.6
  let rb = 0.6
  state {
    regressor perf = 1.0
    ki = 0.0
  }
  events {
    for t in [0.5, 1.0, 1.5, 2.0, 2.5, 3.0] {
      at t {
        set perf = WORST_OF(RATIO(all, reset=false))
        set ki = INDIC(INF("wmin", perf) < kib)
        skip_last receive (cpn if (perf >= cb) & (perf < ab) else 0.0)
        skip_last exercise { forced when perf >= ab; receive 1.0 + cpn }
        only_last receive ((max(perf, 0.0) if ki > 0.5
                            else (1.0 if perf >= rb else max(perf, 0.0)))
                           + (cpn if perf >= cb else 0.0))
      }
    }
  }
}
```

Note the **bitwise** `&` in the coupon gate (so a coupon is not double-paid on the autocall date), the nested **ternary** at maturity, the `INF` running-minimum as the discrete knock-in tracker, and the `for` **loop** keeping the script constant-size over the six observation dates.

5.3 Continuous knock-in — `BARRIER_SURVIVAL` on a fine grid

```
product ContinuousKI {
  currency = "USD"
  discount = 0.02
  underlyings = [SPX]
  let kib = 0.6
  state {
```

```
    regressor perf = 1.0
    ki = 0.0
    surv = 1.0
  }
  events {
    for u in continuous(0.0, 3.0, steps=200) {
      at u { set surv = BARRIER_SURVIVAL("ki", SPX, 60.0, "down",
                                         monitoring="continuous", sigma=0.25) }
    }
    for t in [0.5, 1.0, 1.5, 2.0, 2.5, 3.0] {
      at t {
        set perf = RATIO(SPX, reset=false)
        only_last set ki = 1.0 - surv
        only_last receive (max(perf, 0.0) if ki > 0.5 else 1.0)
      }
    }
  }
}
```

5.4 Arithmetic-average Asian — `RUNNING_MEAN`

```
product Asian {
  currency = "USD"
  discount = 0.02
  underlyings = [SPX]
  state { avg = 100.0 }
  events {
    for t in schedule(start=0.0, end=1.0, freq="1M") {
      at t { set avg = RUNNING_MEAN("avg", SPX) }
    }
    at 1.0 { only_last receive max(avg/100.0 - 1.0, 0.0) }
  }
}
```

5.5 Convertible bond — survival update, `decide` (holder conversion), `IS_ALIVE`, `RECOVERY`

A defaultable coupon bond the holder may convert into `conversion_ratio` shares at any coupon date (American conversion). At each coupon date the script (1) updates the survival indicators, (2) fires a holder `decide` whose immediate value is the conversion proceeds `conversion_ratio * asset` (the holder takes the max of continuing vs converting), (3) pays the coupon if alive, and (4) pays recovery of face on default; survivors redeem face at maturity:

```
product ConvertibleBond {
  currency = "USD"
  underlyings = [ACME_EQ]
  state {
    alive_prev = 1.0
    alive_now = 1.0
    just_defaulted = 0.0
  }
  events {
    for t in [1.0, 2.0, 3.0, 4.0, 5.0] {
      at t {
        set alive_now = alive_prev * IS_ALIVE("ACME")
        set just_defaulted = alive_prev - alive_now
        decide { immediate = alive_now * 2.5 * ACME_EQ + just_defaulted * 1.0 * 2.5 *
ACME_EQ; alive = alive_now > 0; regressors = [ACME_EQ] }
        receive alive_now * 3.0
```

```
        receive just_defaulted * RECOVERY("ACME") * 100.0
        set alive_prev = alive_now
      }
    }
    at 5.0 { receive alive_now * 100.0 }
  }
}
```

(`conversion_ratio = 2.5`, `coupon = 3.0`, `face = 100.0`, `recovery` from the issuer's curve.) This is the exact lowering produced by `make_convertible_bond`.

Part 6 — Grammar & reimplementation reference

This part collects the lexical, grammatical, and tabular facts needed to reimplement the `.amc` language from scratch.

6.1 Lexical conventions

- **Comments:** two forms. `#` starts a comment that runs to the end of the line (a whole line or trailing after code, `currency = "USD"  # deal currency`). `/* … */` is a block comment that may span multiple lines or appear inline between tokens (`x = /* note */ 1.0`). `//` is **not** a comment — it is the floor-division operator (`5 // 2`).
- **Separators:** statements are separated by newlines or `;`. Inside object / leg bodies, `;` or `,` separate fields.
- **Identifiers:** declared names (underlyings, state vars, `let` constants, `groups`, instrument names, string labels) are matched **exactly** as written (case-sensitive).
- **Function names: case-insensitive** (canonicalised to upper case).
- **Structural keywords & flow flags:** fixed lower-case tokens, written exactly.
- **Numbers:** integer or float; percentages may be written `3%` (= 0.03) in settings such as `fixed_rate` and `coupon`.
- **Strings:** double-quoted (`"USD"`, `"2027-01-01"`).
- **Constants:** `True`, `False`, `None`, and the math constants `pi`, `e` are reserved names usable in expressions.

6.2 Dates, tenors, frequencies, day counts

Date atoms (used in `at <date>` slots and `schedule` bounds) are one of:

- a **float year-fraction** — `1.5` (years from `as_of`);
- a **tenor** matching `[+-]?\d+(.\d+)?[DWMYQSA]`, optionally **chained** — `3Y`, `6M`, `1Y+6M`, `1Y6M`, `6M+1D` (chunks summed); a leading `-` marks a **past** date (`-3M`, `-0.5`), used for historical fixings;
- an **ISO calendar date** — `"2027-06-12"`;
- the **name `t0`** (or `as_of`), resolving to year-fraction 0;
- a **bare identifier** — a `let` constant or a `for`-loop variable (e.g. `t`), resolving to that name's date value.

Any of the literal forms may also be **quoted** (`"18M"`, `"1.5"`) for consistency with `as_of`. Tenor units as year-fractions: `D` = 1/365.25, `W` = 7/365.25, `M` = 1/12, `Y` = 1.0; `Q`, `S`, `A` are also accepted as period codes. The parser additionally accepts a parenthesised expression span as a date, but the resolver only reduces it when it is a pure literal/tenor — **arithmetic on a name is not supported in the date slot**: `at t-3M`, `at t + 0.5`, and `at (t - 0.25)` all

fail. Relative offsets must be done inside the payoff expression (`at t { receive f(t - 0.25) }`), not in the `at` slot.

Frequency codes normalise (upper-cased) to ISDA single-letter codes:

- `A` (annual) ← `A`, `Y`, `1Y`, `12M`, `ANNUAL`
- `S` (semi-annual) ← `S`, `6M`, `SEMIANNUAL`
- `Q` (quarterly) ← `Q`, `3M`, `QUARTERLY`
- `M` (monthly) ← `M`, `1M`, `MONTHLY`

Year-fractions per code: `A` = 1.0, `S` = 0.5, `Q` = 0.25, `M` = 1/12.

Day-count / business-day settings (`day_count` / `dcc`, `calendar`, `bda`, `stub`) on `schedule(...)` and swap legs pass through to the engine verbatim (e.g. `"ACT/360"`, `"30/360"`, `"modfollowing"`, `"TARGET"`).

6.3 String-label accumulators

These functions take a **string label as their first argument** and maintain hidden per-path state under that label for the whole simulation, so one call in a `for` loop accumulates across every iteration. Reusing a label reuses the same state; distinct labels are independent:

`BARRIER_SURVIVAL`, `CORRIDOR_SURVIVAL`, `INF`, `PERF_PROD`, `PERF_PROD_IF`, `PERF_SUM`, `PERF_SUM_IF`, `RATIO_PROD`, `RATIO_PROD_IF`, `RATIO_SUM`, `RATIO_SUM_IF`, `REALISED_QUADRATIC_VARIATION`, `REALISED_QUADRATIC_VARIATION_IF`, `REALISED_QV`, `REALISED_QV_CROSS`, `REALISED_QV_IF`, `REALISED_VAR`, `REALISED_VARIANCE`, `REALISED_VARIANCE_IF`, `REALISED_VAR_IF`, `REALISED_VOL`, `REALISED_VOLATILITY`, `REALISED_VOLATILITY_IF`, `REALISED_VOL_IF`, `REALIZED_QUADRATIC_VARIATION`, `REALIZED_QUADRATIC_VARIATION_IF`, `REALIZED_QV`, `REALIZED_QV_CROSS`, `REALIZED_QV_IF`, `REALIZED_VAR`, `REALIZED_VARIANCE`, `REALIZED_VARIANCE_IF`, `REALIZED_VAR_IF`, `REALIZED_VOL`, `REALIZED_VOLATILITY`, `REALIZED_VOLATILITY_IF`, `REALIZED_VOL_IF`, `RUNNING_AVG`, `RUNNING_AVG_IF`, `RUNNING_COUNT`, `RUNNING_COUNT_IF`, `RUNNING_MAX`, `RUNNING_MEAN`, `RUNNING_MEAN_IF`, `RUNNING_MIN`, `RUNNING_PROD`, `RUNNING_PRODUCT`, `RUNNING_PRODUCT_IF`, `RUNNING_PROD_IF`, `RUNNING_SUM`, `RUNNING_SUM_IF`, `RUNNING_SUM_LOG`, `RUNNING_SUM_LOG_IF`, `SHARPE`, `SHARPE_RATIO`, `SHARP_RATIO`, `SNAPSHOT`, `SUP`

(plus the `*_IF` conditional variants and the `REALIZED_*` / `REALISED_*` family, which follow the same label-first convention).

6.4 Expression grammar (complete EBNF)

The expression sub-language is what appears on the right of `receive` / `pay` / `set` and inside function arguments and conditions. It is parsed into a Python-AST subset and validated against the permitted-node whitelist; the resolver then substitutes declared names and re-emits the expression. The grammar, with precedence from lowest (loosest-binding) to highest:

```
expr        ::= ternary

ternary     ::= or_expr [ "if" or_expr "else" ternary ]
                (* element-wise select; compiles to where(cond, a, b) *)

or_expr     ::= and_expr { ( "or" | "|" ) and_expr }
and_expr    ::= not_expr { ( "and" | "&" | "^" ) not_expr }
not_expr    ::= ( "not" | "~" ) not_expr
              | comparison
comparison  ::= add_expr { ( "<" | "<=" | ">" | ">=" | "==" | "!=" ) add_expr }
```

```
add_expr    ::= mul_expr { ( "+" | "-" ) mul_expr }
mul_expr    ::= unary { ( "*" | "/" | "//" | "%" ) unary }
unary       ::= ( "+" | "-" ) unary
              | power
power       ::= postfix [ "**" unary ]          (* right-associative *)
postfix     ::= atom { subscript }
subscript   ::= "[" expr [ ":" expr ] "]"        (* index or slice *)

atom        ::= NUMBER
              | PERCENT                           (* e.g. 3% = 0.03; settings only *)
              | STRING
              | "True" | "False" | "None" | "pi" | "e"
              | NAME                              (* underlying / state / let / group /
const *)
              | call
              | list
              | comprehension
              | "(" expr ")"

call        ::= NAME "(" [ arglist ] ")"
arglist     ::= arg { "," arg } [ "," ]
arg         ::= [ NAME "=" ] expr        (* positional OR a NAME=value keyword arg *)
                  (* keyword args follow any positional ones. The recognised
                     argument NAMES for every built-in (e.g. schedule start/end/
                     freq/day_count, LIBOR currency/t_start/t_end/tenor,
                     CDO_VALUE index/attach/detach/spread/schedule/config,
                     accumulator init/annualization/K/cap/floor/tenor/weights) are
                     listed in the Function Coverage Checklist, "Argument signatures
                     & construct keys" (auto-generated, parse-verified). The grammar
                     accepts any NAME "="; the semantic layer (§6.1.1) rejects keys
                     a given function does not document. *)
              | NAME "=" expr                     (* keyword argument *)

list        ::= "[" [ expr { "," expr } [ "," ] ] "]"
comprehension ::= "[" expr "for" NAME "in" iterable_expr [ "if" expr ] "]"
iterable_expr ::= list | call | NAME

NUMBER      ::= /[+-]?\d+(\.\d+)?([eE][+-]?\d+)?/
PERCENT     ::= NUMBER "%"
STRING      ::= '"' ... '"'
NAME        ::= /[A-Za-z_][A-Za-z0-9_]*/   (* but not a reserved keyword *)

(* Globally reserved keywords (never usable as value names): at, autocall, bond,
   by, callable, cancel, CDO, convertible, decide, enter, events, exercise, fix,
   for, in, knock_in, knock_out, let, mutable, pay, pay_undisc, pricing, product,
   putable, receive, receive_undisc, schedule, schedule_div, set, state, swap,
   tranche_pricer, when, and
   the flow flags skip_first/skip_last/only_first/only_last. (Note `pay` is also a
   leg-direction word inside swap bodies.)
   Context-local words — special only in a specific position, otherwise ordinary
   names: forward, currency, fx (cashflow suffix); regressor (state block);
   continuous (sequence); while, into, issuer, forced (exercise fields);
   convert, call, put, reset, change_of_control, contingent, soft, hard
   (convertible blocks). These may still be used as state-variable names.
   The constants True/False/None/pi/e are reserved value names. Function names
   (Part 3/§6.6) are recognised in call position only and are case-insensitive.
   There is no user-facing `do` or `flow` keyword: `do` is an internal field the
   resolver emits for state updates (write `set name = expr` instead), and `flow`
   is an internal transcription node — both may be used as ordinary names. *)
```

Resolution rules the reimplementation must apply (after parsing an expression):

1. **Function calls.** `NAME` in call position must (case-insensitively) be one of the built-in functions (Part 3 / §6.6). Canonicalise the name to upper case. Validate the **positional**

argument count against the arity table (§6.6); keyword arguments are extra. Only the functions/kwargs documented as accepting keywords may receive them — the recognised argument **names** for every built-in are listed in the Function Coverage Checklist, "Argument signatures & construct keys" (auto-generated from the engine and parse-verified).

2. **Names in value position** resolve, in order, to: a comprehension-bound variable (local), a declared **underlying** (→ its simulated value at the current event), a **`state`** variable, a **`let`** constant (inlined as a literal), a **static vector** (`let`/`mutable` list — a bare reference expands to its literal list; a subscript `v[i]` with a compile-time index folds to the element), a **compile-time `mutable`** scalar (inlined as its current literal value at this point in the lowering walk), a declared **`groups`** name (a basket, valid only as an aggregator argument), a **constant** (`pi`, `e`, `True`, `False`, `None`), or an **instrument** name (valid only inside an event, lowered to its inner-value call). Anything else is an "unknown name" error.
3. **Static-vector subscripts and `SIZE`.** `v[i]` / `v[i][j]` where `v` is a `let`/`mutable` vector and `i` (`j`) folds to an integer → the literal element (out-of-range index is a compile error). `SIZE(x)` / `LEN(x)` / `LENGTH(x)` fold to the compile-time integer length of `x`; `NROWS(x)` / `NCOLS(x)` to the row / first-row length of a nested vector. `x` may be a `let`/`mutable` vector, a literal list, a `groups` name, a **`let`-bound `schedule(...)`** (its generated date count), or a **`state` array** name (its declared slot count). `SIZE` of a `schedule_div(...)` or of any runtime per-path value is a compile error (the length is not known at lowering). `SIZE(...)` may be used as a `range` bound, so `for j in range(0, SIZE(v))` stays length-safe.
4. **`RATIO/PERF` with `reset=false`** (case-insensitive) are rewritten by a pre-pass into the fixed-inception reference-ratio form, distributing over a group argument; `reset=true` keeps the per-call reset.
5. **Compile-time mutables.** A `mutable` is seeded at its declaration and updated in place by each `set <mutable> = <const-expr>` as the resolver walks events in order; the current value is substituted into every expression that references it. The RHS (and any slot index) must fold to a compile-time constant — a runtime/per-path RHS is an error. Mutables vanish after lowering (like `func` inlining); they never appear in the resolved product.
6. **Forbidden nodes** (compile error): attribute access (`a.b`), lambdas, assignments, comprehension over anything but a list/call/name, walrus, f-strings, imports, starred args, dict/set literals in expressions.
7. **Element-wise semantics.** Every operator and function is element-wise over the per-path value vector; boolean `&`/`|`/`^`/`~` are the element-wise mask operators (the keyword `and`/`or`/`not` short-circuit and are for scalars).

6.5 Program grammar (complete EBNF)

The lexical layer (applied before the grammar below) is: skip whitespace and newlines; skip `#`-to-end-of-line comments; skip `/* … */` block comments (which may span lines); `//` is **not** a comment (it is the floor-division operator). Tokens are `NAME`, `NUMBER`, `STRING` (double-quoted), `PERCENT` (`NUMBER "%"`), `TENOR`, the punctuation `{ } [ ] ( ) = ; ,`, and the reserved keywords. Statements are separated by newlines or `;`; object/leg fields by `;` or `,`. A `product` block is the top-level unit.

```
program       ::= "product" NAME "{" { product_item } "}"

product_item  ::= setting
                | binding
                | mutable_decl
```

```
                | group_decl
                | func_decl
                | state_block
                | instrument
                | events_block

(* ---- user-defined value functions (inlined at compile time) ---- *)
func_decl     ::= "func" NAME "(" [ params ] ")" "{" func_body "}"
params        ::= param_decl { "," param_decl }
param_decl    ::= NAME [ "=" expr ]                 (* trailing default *)
func_body     ::= { func_stmt } return_stmt
                  (* every path must end in a return; for-bodies are let/for only *)
func_stmt     ::= let_stmt
                | for_stmt
                | if_stmt
let_stmt      ::= "let" NAME "=" expr
for_stmt      ::= "for" NAME "in" iterable "{" { let_stmt | for_stmt } "}"
                  (* compile-time unrolled; range/list bound must resolve to an
                     integer at compile time (a `let` is allowed) *)
iterable      ::= list | "range" "(" expr [ "," expr [ "," expr ] ] ")" | NAME
if_stmt       ::= "if" expr "{" func_body "}" [ "else" "{" func_body "}" ]
                  (* lowers to a vectorised ternary `then if cond else else` *)
return_stmt   ::= "return" expr

(* ---- settings & declarations ---- *)
setting       ::= NAME "=" setting_value          (* any non-keyword top key *)
                  (* recognised keys: currency, discount, discount_rate, as_of,
                     underlyings, pricing, plus any setting-style key.
                     pricing = closedform | closeform | analytic — selects the CCR
                     valuation route for a CDO (Part 7 §7.4 / Execution Spec §9):
                     closedform attaches the analytic pricer and bypasses LSM
                     regression (guarded against path-gated tranches); analytic is
                     advisory and stays on the control-variate regression. *)
setting_value ::= STRING | NUMBER | list | object | expr
binding       ::= "let" NAME [ shape ] "=" ( schedule_call | value | list )
                  (* a list value (optionally shaped) declares a STATIC, immutable
                     compile-time vector: `let w = [0.2, 0.3, 0.5]`,
                     `let strikes[3] = [90, 100, 110]`,
                     `let grid[2][3] = [[1,2,3],[4,5,6]]` (nested). The optional
                     shape is asserted against the literal at compile time. Elements
                     are constant expressions, folded at lowering. A subscript
                     `w[i]` with a compile-time-foldable index folds to the literal
                     element; a bare `w` used as a list argument expands to the
                     literal list. *)
shape         ::= "[" INT "]" { "[" INT "]" }       (* fixed dimension sizes *)

mutable_decl  ::= "mutable" NAME [ shape ] "=" ( value | list )
                  (* a COMPILE-TIME-mutable scalar (a counter) or vector. Unlike
                     `let` it is reassignable via `set` (below); unlike `state` it
                     is NOT per-path — it is a single value resolved entirely at
                     lowering and never reaches the engine. Its assignment RHS must
                     be compile-time constant (it may reference `let`, other
                     `mutable`s, `SIZE(...)`, and arithmetic — but NOT an underlying,
                     `state`, `t`, or any per-path value). It may be assigned where
                     its loop unrolls at lowering: top level, a `for v in [<list>]`
                     /range loop, or a `for v in schedule(...)` with compile-time
                     dates (plain start/end/freq or dates=[...], no calendar/bda) —
                     which then unrolls per date. NOT inside schedule_div or a
                     calendar-rolled schedule, nor under a runtime `if`. Use `state`
                     for a per-path counter. *)
group_decl    ::= ( "groups" | "baskets" | "group" | "basket" ) NAME
                  "=" "[" NAME { "," NAME } "]"

state_block   ::= "state" "{" { state_entry } "}"
state_entry   ::= [ "regressor" ] NAME [ "[" INT "]" ] "=" expr
```

```
                  (* "[" INT "]" declares constant-size array state *)

(* ---- instruments (physical-settlement deliverables) ---- *)
instrument    ::= swap_decl | bond_decl | forward_decl | convertible_decl
                | tranche_pricer_decl | cdo_decl

(* ---- credit tranche instruments (see Part 7) ---- *)
tranche_pricer_decl ::= "tranche_pricer" NAME "{" { setting } "}"
                  (* keys: el_mode (fast|exact), recovery_states (2|3), gh_nodes,
                     grid, maturity, recovery_markdown, recovery_levels,
                     recovery_var_frac *)
cdo_decl      ::= "CDO" NAME "{" { setting } "}"
                  (* keys: index (STRING), attach, detach, spread,
                     schedule (a schedule(...) call — endpoint inclusive),
                     config (a tranche_pricer NAME), upfront (None|par|NUMBER),
                     IsUpfrontPayer, nominalExchange, amortizing. Entered with
                     `enter <NAME>` (effective date, no cashflow); referencing
                     <NAME> in an expression lowers to CDO_VALUE(...). *)

swap_decl     ::= "swap" NAME "{" { swap_item } "}"
swap_item     ::= setting                          (* maturity, tenor, fixed_rate,
                                                      frequency, pay, float,
                                                      float_spread, cms_tenor,
                                                      cms_freq, notional, currency *)
                | leg
leg           ::= ( "pay" | "receive" ) leg_type "{" { setting } "}"
leg_type      ::= "fixed" | "float" | "cms" | "cms_spread"
                  (* leg settings: frequency, notional, currency,
                     exchange_notional; fixed→rate; float→index|forward, spread,
                     fixing(in_advance|in_arrears); cms→swap_tenor, index, spread,
                     swap_freq, swap_day_count; cms_spread→swap_tenor, swap_tenor2 *)

bond_decl     ::= "bond" NAME "{" { setting } "}"
                  (* keys: maturity, coupon (alias fixed_rate), frequency,
                     notional (alias face_value), issuer, recovery *)
forward_decl  ::= "forward" NAME "{" { setting } "}"
                  (* keys: underlying | pair, strike, maturity, long, notional *)

convertible_decl ::= "convertible" NAME "{" { conv_setting | conv_block } "}"
conv_setting  ::= NAME "=" value
                  (* known flat settings (any other key is an error): underlying,
                     notional, maturity, coupon, frequency, ratio, currency,
                     issuer, recovery, start, settlement, mandatory_conversion *)
conv_block    ::= ( "convert" | "call" | "put" | "reset" | "change_of_control" )
                  "{" { conv_field | conv_clause } "}"
conv_field    ::= NAME "=" value
                  (* e.g. window=[from,to], schedule=[…], price=<pct>,
                     dates=[…], floor=<expr>, hard, make_whole=<bool> *)
conv_clause   ::= ( "contingent" | "soft" | "hard" ) raw_expr
                  (* e.g. `contingent when U >= 1.3 * conv_price`;
                     `soft when U >= K for N of last M days`; `hard from <date>`.
                     `conv_price` is a reserved name inside convertible clauses. *)

(* ---- events ---- *)
events_block  ::= "events" "{" { event } "}"

event         ::= at_event
                | for_loop
                | schedule_block
                | generic_event
                | exercise_event
                | bare_stmt                          (* runs at t0 *)

at_event      ::= "at" date ( stmt_block | stmt )
for_loop      ::= "for" NAME "in" sequence stmt_block_events
```

```
                  (* the preferred form for schedule / schedule_div / continuous;
                     binds NAME to each generated date. For schedule_div the body
                     may use the bare tokens `current` (0-based dividend index) and
                     `SIZE` (dividend count), substituted per dividend by the
                     engine at expansion time. *)
schedule_block::= ( "schedule" | "schedule_div" ) "(" sched_args ")"
                  "{" { event } "}"
                  (* standalone (no loop variable) shorthand for the for_loop
                     above; same expansion, but no date variable is bound *)
generic_event ::= generic_type "{" { field } "}"
generic_type  ::= "american" | "convertible" | "pay_undisc"
                | "bond_physical_exercise" | "swap_physical_exercise"
                | "fx_physical_exercise" | "equity_physical_exercise"

sequence      ::= list                              (* explicit date list *)
                | "schedule" "(" sched_args ")"
                | "schedule_div" "(" sched_args ")" (* dividend-driven; binds the
                                                       loop var to t_ex−1e-9 (ex_date)
                                                       or t_pay (pay_date); body may
                                                       use bare `current` / `SIZE` *)
                | "continuous" "(" date "," date [ "," "steps" "=" INT ] ")"
                | NAME                              (* a let-bound sequence *)

sched_args    ::= sched_kw { "," sched_kw } [ "," ]
sched_kw      ::= sched_key "=" sched_val
sched_key     ::= "start" | "end" | "n" | "freq" | "frequency" | "stub"
                | "dates" | "calendar" | "bda" | "day_count" | "dcc"
                | "underlying" | "include_first" | "include_last"
                | "drive_by" | "obs_time"           (* schedule_div extras *)

stmt_block        ::= "{" { stmt } "}"
stmt_block_events ::= "{" { event } "}"

(* ---- statements (inside at / for / schedule bodies) ---- *)
stmt          ::= { flow_flag } stmt_body
flow_flag     ::= "skip_first" | "skip_last" | "only_first" | "only_last"

stmt_body     ::= receive_stmt
                | set_stmt
                | exercise_stmt
                | knock_stmt
                | enter_stmt
                | fix_stmt
                | generic_event
                (* NB: state updates use `set name = expr`. There is no
                   user-facing `do` statement — `do` is an internal field the
                   resolver emits when lowering state updates, not a surface
                   keyword. *)

receive_stmt  ::= cashflow_kw expr [ ccy_suffix ]      (* bare-expression form *)
                | cashflow_kw object                   (* faithful block form *)
cashflow_kw   ::= "receive" | "pay" | "receive_undisc" | "pay_undisc"
ccy_suffix    ::= "currency" STRING [ "fx" STRING ]    (* cross-currency / quanto *)
                  (* In the block form the same two keys appear as object fields:
                     `currency = "EUR"` and optional `fx = "EURUSD"`.
                     Sign: receive(+) / pay(−); discounting: plain(yes) /
                     *_undisc(no). The faithful block's `amount` is literal — it is
                     NOT re-signed (the printer round-trips dict pay events this
                     way); only the bare `pay`/`pay_undisc` keyword applies the
                     holder-pays negation. *)
set_stmt      ::= "set" NAME [ "[" expr "]" ] "=" expr  (* "[" expr "]" = array slot *)
                  (* If NAME is a `state` variable, a per-path runtime update; if a
                     `mutable`, a compile-time update (the slot index and RHS must
                     fold to compile-time constants, and it emits no runtime event).
                     A `set` after a `receive` in the same block runs AFTER the
```

```
                    payment (the payment sees the pre-update value); equivalently
                    a fix carries do_when ∈ {before, after}, default before. *)
knock_stmt    ::= "knock_in" NAME "when" expr
                | "knock_out" "when" expr [ "rebate" expr ]
enter_stmt    ::= ( "enter" | "cancel" ) NAME [ "when" expr | by_block ]
                  (* three forms: bare `enter S` (unconditional, latched at the
                     event date), `enter S when <cond>` (wired trigger), and
                     `enter S { by <party> }` (option attributed to a party).
                     `when` and the by-block are mutually exclusive.
                     For swap / bond / forward instruments this is physical
                     settlement (the deliverable is entered/cancelled). For a
                     `convertible`, only the bare `at <date> enter <name>` form is
                     valid and it sets the convertible's ISSUE DATE (equivalent to
                     `start = <date>`); `cancel` and `when`/by-block are rejected
                     for convertibles. *)
by_block      ::= "{" "by" party "}"
party         ::= "bank" | "counterparty"
fix_stmt      ::= "fix" [ object ]

exercise_stmt ::= "exercise" "{" { exercise_field } "}"
                | "autocall" object        (* keys: when, pay *)
                | "callable" object        (* keys: pay (required), when *)
                | "putable" object         (* keys: pay (required), when *)
                | "decide" object          (* keys: immediate, alive, forced /
                                              forced_stop, continuation,
                                              regressors, polarity, controls;
                                              also accepts when / pay *)
exercise_field::= "forced" "when" expr
                | "receive" expr
                | "into" NAME
                | "issuer"
                | "by" party
                | "while" expr

exercise_event::= "exercise" NAME stmt_block      (* physical macro form *)

(* ---- shared ---- *)
date          ::= NUMBER                    (* year-fraction *)
                | TENOR                     (* 3Y, 6M, 1Y+6M, -3M (past) *)
                | STRING                    (* ISO date or quoted tenor/number *)
                | "t0"                      (* = as_of, year-fraction 0 *)
                | NAME                      (* let constant or for-loop var *)
                (* NB: no arithmetic in this slot — `t-3M`, `t+0.5` do NOT parse;
                   do relative offsets inside the payoff expression instead *)
TENOR         ::= /[+-]?\d+(\.\d+)?[DWMYQSA]([+-]?\d+(\.\d+)?[DWMYQSA])*/
                | NAME                      (* t0 / as_of *)
object        ::= "{" [ field { ( ";" | "," ) field } ] "}"
field         ::= NAME "=" value
value         ::= STRING | NUMBER | PERCENT | list | object | expr
list          ::= "[" [ value { "," value } ] "]"
raw_expr      ::= /* a raw expression span captured verbatim up to a depth-0
                     terminator (newline / ; / }) — used by convert/soft/hard
                     clauses and conditions; validated as an expr at resolve time */
INT           ::= /\d+/
```

Lowering rules the reimplementation must apply (parsed tree → canonical script dict consumed by the pricer):

1. **`for` loops** are unrolled: the body events are emitted once per date in the sequence, with the loop variable bound to each date; flow flags (`skip_first` / `only_last` / …) select which iterations a statement fires on.
2. **Adjacent `receive`/`pay`** at one instant **net into a single discounted payment** (`type: "pay"`, summed amount, `pay` negated). `receive_undisc` / `pay_undisc` each emit a

separate `pay_undisc` event (`discounted=False`, `pay_undisc` negated) — they never merge into the discounted accumulator.

3. **A `set` written after a payment** is ordered to run *after* that payment's amount is computed (so the amount sees the pre-update state); a `set` with no preceding payment becomes a leading pre-update.
4. **`exercise { receive max(CV,X) }`** (Bellman form) lowers to a native `american` event with the regressor auto-inferred; `forced when` adds the mandatory-stop condition; `issuer` sets `polarity="min"`; `into NAME` is parsed but **not yet lowered** (raises; use `enter`).
5. **`callable`/`putable`** desugar to a `decide` against the call/put price with `polarity` `min`/`max`; **`autocall`** is a forced (non-optimised) stop.
6. **`knock_in`/`knock_out`** latch a per-path flag (and optional `rebate` cashflow); the engine handles path termination.
7. **`schedule`/`schedule_div`** expand to one event per generated date; `schedule_div` additionally binds the dividend macros (`DIV_AMOUNT`, `DIV_C`, `DIV_Y`, …) inside its body and requires `market_config`.
8. **Same-instant events** are ordered by a `(date, seq)` tuple; the printer reproduces this with epsilon date offsets on round-trip.
9. **Tenors / frequencies / percentages** are normalised per §6.2 before emission (tenor → year-fraction, frequency → ISDA code, `3%` → `0.03`).
10. **Cross-currency cashflows.** A `receive`/`pay`/`receive_undisc`/`pay_undisc` carrying `currency` (≠ the product currency) is emitted as a standalone event (it cannot merge into the single-currency discounted accumulator) with `currency` (and optional `fx`) attached; `pay`/`pay_undisc` negate the amount, `receive`/`receive_undisc` keep it positive. The engine multiplies by spot FX at the event date and discounts on the domestic curve.
11. **`convertible NAME { … }`** lowers to risky/risk-free coupon + redemption `pay` events plus the windowed convert/call/put/reset decision events; inside its clauses `conv_price` is a reserved name. `coupon` accepts a percent literal; `issuer`+`recovery` make the coupons survival-weighted. The whole instrument self-expands (no `events` block needed); its issue date is `t0` unless set by a `start =` setting or an `at <date> enter NAME` statement (equivalent; the `enter` form overrides `start`), which shifts the entire coupon/window/schedule timeline.
12. **`func NAME(params) { … }`** is inlined at every call site and then disappears: the body is lowered to a single expression by back-substituting params and locals, unrolling compile-time `for` loops into folds, and rewriting `if`/`else` into vectorised ternaries (`a if c else b`). Because nothing survives lowering, funcs never appear in the canonical dict and do not affect round-trip. Funcs are pure (no events, no `set`); recursion and built-in-name shadowing are compile errors.

Together with the operator set (Part 1), the function vocabulary with signatures and kwargs (Part 3), and the arity table (§6.6), these two grammars plus the lowering rules fully specify `compile_amc`: parse → resolve/validate names and functions → lower events → emit the canonical script dict.

6.6 Function arity table

Minimum–maximum **positional** argument counts for every built-in (canonical upper-case names; `N+` means N-or-more / variadic). Keyword arguments are additional and not counted here.

Function	Args	Function	Args
`ABS`	1–2	`ABSOLUTE_BASKET`	2–3

Function	Args	Function	Args
ALL	0+	ANY	0+
ARGMAX_OF	0+	ARGMIN_OF	0+
AVERAGE	0+	AVG	0+
AVG_EXCLUDING	2–3	BACHELIER_EURO_CALL	5
BACHELIER_EURO_PUT	5	BARRIER_SURVIVAL	4–12
BASKET	1+	BASKET_LOSS	1–2
BEST	0+	BEST_OF	0+
BEST_OF_EXCLUDING	2–3	BEST_OF_IDX	0+
BEST_OF_IDX_EXCLUDING	2–3	BOND	0+
BS_EURO_CALL	6	BS_EURO_PUT	6
CALL	2	CALL_PAYOFF	2
CALL_SPREAD	2–4	CAPPED_AT	2
CAP_AT	2	CEIL	1–2
CLAMP	3	CLIP	1–4
CMS	3–5	COLLAR	3
CORRIDOR_SURVIVAL	4–13	COUNT	0+
DEFAULTED	1	DF	2
DIGITAL	1	DIV_AMOUNT	1
DIV_C	0–1	DIV_T_EX	0–1
DIV_T_PAY	0–1	DIV_Y	0–1
DOUBLE_DIGITAL	3	EXCLUDE_CONT_BEST	0+
EXCLUDE_CONT_WORST	0+	EXP	1–2
EXPM1	1–2	FIRST_TO_DEFAULT	1
FIX	2+	FIXING	1–2
FLOOR	1–2	FLOORED_AT	2
FLOOR_AT	2	FORWARD	2
FWD_DIV_AMOUNT	1	FWD_SURVIVAL	2
FWD_SURVIVAL_BOND	2	HAZARD	1
HEAVISIDE	1	IF	1–3
IFELSE	1–3	IF_	1–3
INDEX_SPREAD_NOTIONAL	1–2	INDIC	1
INDICATOR	1	INDICATOR_OF	1
INF	2–3	INTENSITY	1
INTERP	3	IS_ALIVE	1
JUST_DEFAULTED	2	KTH_BEST	1+
KTH_WORST	1+	LADDER	3
LERP	3	LIBOR	3–4
LINTERP	3	LN	1–2
LOG	1–2	LOG10	1–2
LOG1P	1–2	LOG2	1–2
MAX	0+	MAXIMUM	0+
MEAN	0+	MEDIAN	0+
MEDIAN_EXCLUDING	2–3	MIN	0+
MINIMUM	0+	NEG	1
NEGATIVE_PART	1	NTH_BEST	1+

Function	Args	Function	Args
NTH_WORST	1+	NUM_DEFAULTED	1
OIS	3	PAST_FIX	1–2
PCTRANK	1+	PCT_RANK	1+
PERCENTILE	1+	PERCENTILE_RANK	1+
PERF	2–5	PERF_ACCRUED	2–5
PERF_PROD	3–6	PERF_PROD_IF	4–7
PERF_SUM	3–6	PERF_SUM_IF	4–7
PIECEWISE	3	POS	1
POSITIVE_PART	1	POW	2–3
PROD	0+	PRODUCT	0+
PUT	2	PUT_PAYOFF	2
QUANTILE	1+	Q_SURVIVE	2
Q_SURVIVE_BOND	2	RAMP	1
RANGE	0+	RANGE_DIGITAL	3
RANK	1+	RANK_OF	1+
RATIO	1–3	RATIO_ACCRUED	1–3
RATIO_PROD	2–4	RATIO_PROD_IF	3–5
RATIO_SUM	2–4	RATIO_SUM_IF	3–5
REALISED_QUADRATIC_VARIATION	2	REALISED_QUADRATIC_VARIATION_IF	3
REALISED_QV	2	REALISED_QV_CROSS	3
REALISED_QV_IF	3	REALISED_VAR	2–3
REALISED_VARIANCE	2–3	REALISED_VARIANCE_IF	3–4
REALISED_VAR_IF	3–4	REALISED_VOL	2–3
REALISED_VOLATILITY	2–3	REALISED_VOLATILITY_IF	3–4
REALISED_VOL_IF	3–4	REALIZED_QUADRATIC_VARIATION	2
REALIZED_QUADRATIC_VARIATION_IF	3	REALIZED_QV	2
REALIZED_QV_CROSS	3	REALIZED_QV_IF	3
REALIZED_VAR	2–3	REALIZED_VARIANCE	2–3
REALIZED_VARIANCE_IF	3–4	REALIZED_VAR_IF	3–4
REALIZED_VOL	2–3	REALIZED_VOLATILITY	2–3
REALIZED_VOLATILITY_IF	3–4	REALIZED_VOL_IF	3–4
RECOVERY	1	RELATIVE_BASKET	2–4
RFR	3–4	RMS	0+
RUNNING_AVG	2–3	RUNNING_AVG_IF	3–4
RUNNING_COUNT	1–2	RUNNING_COUNT_IF	2–3
RUNNING_MAX	2–3	RUNNING_MEAN	2–3
RUNNING_MEAN_IF	3–4	RUNNING_MIN	2–3
RUNNING_PROD	2–3	RUNNING_PRODUCT	2–3
RUNNING_PRODUCT_IF	3–4	RUNNING_PROD_IF	3–4
RUNNING_SUM	2–3	RUNNING_SUM_IF	3–4
RUNNING_SUM_LOG	2–3	RUNNING_SUM_LOG_IF	3–4
SHARPE	2–4	SHARPE_RATIO	2–4
SHARP_RATIO	2–4	SIGMOID	1–2
SIGN	1–2	SMOOTH_INDICATOR	2–5
SMOOTH_STEP	2–5	SNAPSHOT	2–3

Function	Args	Function	Args
SOFTMAX	1+	SOFTMIN	1+
SOFT_MAX	1+	SOFT_MIN	1+
SQRT	1–2	STD	0+
STDDEV	0+	STDDEV_EXCLUDING	2–3
STDEV	0+	STDEV_EXCLUDING	2–3
STD_EXCLUDING	2–3	STEP	1
STRADDLE	2	SUM	0+
SUM_BEST	1+	SUM_BEST_K	1+
SUM_EXCLUDING	2–3	SUM_FWD_DIVS	3
SUM_LOG	0+	SUM_REALIZED_DIVS	3
SUM_WORST	1+	SUM_WORST_K	1+
SUP	2–3	SWAP	1–2
TRANCHE_LOSS	3–5	VAR	0+
VARIANCE	0+	VARIANCE_EXCLUDING	2–3
VAR_EXCLUDING	2–3	WHERE	1–3
WORST	0+	WORST_OF	0+
WORST_OF_EXCLUDING	2–3	WORST_OF_IDX	0+
WORST_OF_IDX_EXCLUDING	2–3	YOY_INFLATION	3
ZC_INFLATION_SWAP_RATE	3		

Part 7 — Credit tranche instruments & pricing modes

Parts 1–6 cover payoffs whose cashflows are written out in `events`. A **CDO tranche** is instead declared as a first-class *instrument block* and valued by the analytic credit kernel; the script only enters it and books its value.

7.1 The `tranche_pricer` config block

A reusable pricer configuration (the `TranchePricerConfig` fields):

```
tranche_pricer tp { el_mode = fast; recovery_states = 2; gh_nodes = 32; grid = 160 }
```

`el_mode` = `fast` (one bucket recursion at maturity + loss-clock interpolation per horizon) or `exact` (a recursion per horizon); `recovery_states` = `2` (deterministic LGD-sized loss bucket) or `3` (stochastic recovery with `recovery_var_frac`); `gh_nodes` Gauss–Hermite factor nodes; `grid` loss buckets.

7.2 The `CDO` instrument block

```
CDO C { index = "CDX"; attach = 0.10; detach = 0.30; spread = 0.05
        schedule = schedule(start=3M, end=5Y, freq=6M, day_count="ACT/365")
        config = tp; upfront = None; IsUpfrontPayer = True; nominalExchange = False }
```

Keys: `index` (credit index name), `attach`/`detach` (tranche strikes), `spread`, `schedule` (premium leg dates — endpoint inclusive), `config` (a `tranche_pricer`), `upfront` (`None` | `par` | float), `IsUpfrontPayer`, `nominalExchange`, and an optional amortisation list. Premium and protection legs both live **inside** the closed form; you do not write them out.

7.3 `enter` and `CDO_VALUE` (`receive C`)

`enter C` marks the tranche's **effective date** — the date it comes into force — and emits no cashflow of its own. Referencing the instrument name in an expression lowers to `CDO_VALUE(...)`, the analytic tranche value (protection-buyer canonical, per unit); before the effective date it is zero.

```
events { at 1M { enter C }            # effective date
         at 5Y { receive C } }        # receive C  ->  CDO_VALUE(...) closed form
```

7.4 `pricing = closedform | analytic`

A product-level directive selecting the counterparty-exposure valuation route:

```
pricing = closedform
```

`closedform` (alias `closeform`) attaches an analytic pricer so the forward mark-to-market is the closed form evaluated directly on the P-measure paths — **no Longstaff–Schwartz regression on Q** — and is *guarded*: if the CDO value is combined with a path-state gate (a knock-out `step(IS_ALIVE …)`, inline or routed through a `set` variable), compilation is refused, because the closed form cannot see the gate. `pricing = analytic` is a **separate, advisory** value that keeps the deal on the control-variate regression route. Use `closedform` only for un-gated tranches; for gated / knock-out tranches leave it off and let the (auto-wired control-variate) regression handle them. The execution details are in the Execution Spec §9.

Appendix — Keyword & function index

Every operator, keyword, flag, and built-in function, each linked to the section where it is defined; the page number after each entry is the page of that definition (click to jump).

Operators

`x if c else y (ternary)`
`[…] list / x[i] index / slice`
`[e for x in xs] comprehension`

Structural keywords

`product`
`currency`
`discount`
`discount_rate`
`as_of`
`t0`
`underlyings`
`let`
`state`
`CV`
`CONTINUATION_VALUE`
`EXERCISED`
`HIT`
`regressor`
`groups`
`events`
`at`
`for`
`in`
`schedule (iterator)`
`continuous (iterator)`
`receive`
`pay`
`set`
`knock_in`
`knock_out`
`rebate`
`enter`
`cancel`
`exercise`
`autocall`
`callable`
`putable`
`decide`
`convertible`
`forced`
`when`
`issuer`
`by`
`into`
`while`
`polarity`
`regressors`
`swap`
`bond`
`forward`

`receive_undisc`
`pay_undisc`

Flow flags

`skip_first`
`skip_last`
`only_first`
`only_last`

Built-in functions (A–Z)

`abs`
`ABSOLUTE_BASKET`
`ALL`
`american`
`ANY`
`ARGMAX_OF`
`ARGMIN_OF`
`AVERAGE`
`AVG`
`AVG_EXCLUDING`
`BACHELIER_EURO_CALL`
`BACHELIER_EURO_PUT`
`BARRIER_SURVIVAL`
`BASKET`
`BASKET_LOSS`
`BEST_OF`
`BEST_OF_EXCLUDING`
`BEST_OF_IDX`
`BEST_OF_IDX_EXCLUDING`
`BOND`
`bond_physical_exercise`
`BS_EURO_CALL`
`BS_EURO_PUT`
`call_payoff`
`call_spread`
`cap_at`
`ceil`
`clamp`
`clip`
`CMS`
`collar`
`CORRIDOR_SURVIVAL`
`COUNT`
`DEFAULTED`
`DF`
`digital`
`DIV_AMOUNT`
`DIV_C`
`DIV_T_EX`
`DIV_T_PAY`

DIV_Y
double_digital
equity_physical_exercise
EXCLUDE_CONT_BEST
EXCLUDE_CONT_WORST
exp
expm1
FIRST_TO_DEFAULT
FIX
FIXING
floor
floor_at
FORWARD
FWD
FWD_DIV_AMOUNT
FWD_SURVIVAL
FWD_SURVIVAL_BOND
fx_physical_exercise
HAZARD
heaviside
if_
ifelse
INDEX_SPREAD_NOTIONAL
indic
indicator
INF
INTENSITY
INTERP
IS_ALIVE
JUST_DEFAULTED
KTH_BEST
KTH_WORST
ladder
LERP
LIBOR
LINTERP
ln
log
log10
log1p
log2
max
MEAN
MEDIAN
MEDIAN_EXCLUDING
min
neg
negative_part
NTH_BEST
NTH_WORST
NUM_DEFAULTED
OIS

PAST_FIX
PCT_RANK
PERCENTILE
PERCENTILE_RANK
PERF
PERF_ACCRUED
PERF_PROD
PERF_PROD_IF
PERF_SUM
PERF_SUM_IF
piecewise
pos
positive_part
pow
PROD
PRODUCT
put_payoff
Q_SURVIVE
Q_SURVIVE_BOND
QUANTILE
ramp
RANGE
range_digital
RANK
RANK_OF
RATIO
RATIO_ACCRUED
RATIO_PROD
RATIO_PROD_IF
RATIO_SUM
RATIO_SUM_IF
REALISED_QV
REALISED_QV_CROSS
REALISED_VAR
REALISED_VOL
REALIZED_QV
REALIZED_QV_CROSS
REALIZED_QV_IF
REALIZED_VAR
REALIZED_VAR_IF
REALIZED_VOL
REALIZED_VOL_IF
RECOVERY
RELATIVE_BASKET
RFR
RMS
RUNNING_AVG
RUNNING_COUNT
RUNNING_COUNT_IF
RUNNING_MAX
RUNNING_MEAN
RUNNING_MEAN_IF

RUNNING_MIN
RUNNING_PROD
RUNNING_PROD_IF
RUNNING_PRODUCT
RUNNING_SUM
RUNNING_SUM_IF
RUNNING_SUM_LOG
RUNNING_SUM_LOG_IF
SHARP_RATIO
SHARPE
SHARPE_RATIO
sigmoid
sign
smooth_indicator
smooth_step
SNAPSHOT
soft_max
soft_min
softmax
softmin
sqrt
STD
STDDEV
STDEV
STDEV_EXCLUDING
step
straddle
SUM
SUM_BEST_K
SUM_EXCLUDING
SUM_FWD_DIVS
SUM_LOG
SUM_REALIZED_DIVS
SUM_WORST_K
SUP
SWAP
swap_physical_exercise
TRANCHE_LOSS
VAR
VAR_EXCLUDING
VARIANCE
where
WORST_OF
WORST_OF_EXCLUDING
WORST_OF_IDX
WORST_OF_IDX_EXCLUDING
YOY_INFLATION
ZC_INFLATION_SWAP_RATE